SENG 31242

Scooter Rent Kandy

By

Jojeevan – SE/2021/013

Kajanan – SE/2021/032

Gobinath – SE/2021/045

Mohamad – SE/2021/048

A report submitted in partial fulfilment of the requirements for the degree of

Bachelor of Science Honours in Software Engineering (B.Sc.SE)

Software Engineering Teaching Unit

Faculty of Science

University of Kelaniya

Sri Lanka

2026

Deployed System: http://65.0.5.214/

DECLARATION

I hereby certify that this project and all the artifacts associated with it is my own work and it has not been submitted before nor is currently being submitted for any other degree program.

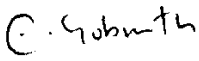
Full name of the student: Jojeevan
Student No: SE/2021/013

Signature of the student:  Date: 25/10/2026

Full name of the student: Kajanan Sivalingam
Student No: SE/2021/032

Signature of the student:  Date: 25/10/2026

Full name of the student: Gobinath Ehamparam
Student No: SE/2021/045

Signature of the student:  Date: 25/10/2026

Full name of the student: Mohamed Riyal Mohamed
Student No: SE/2021/048

Signature of the student:  Date: 25/10/2026

Name of the supervisor: Mrs. Nimesha Arambepola

Signature of the supervisor: Date: 25/10/2026

ACKNOWLEDGEMENTS

We would like to express our sincere thanks to the Software Engineering Teaching Unit (SETU) for providing us the opportunity to work on the Scooter Rental Kandy project. This experience allowed us to apply our academic knowledge to a real-world application while gaining valuable hands-on experience in software development.

We also extend our gratitude to the judging panel for their insightful feedback during the initial idea pitching, which helped shape and improve the concept of our project. Special thanks to our batch mates for their constant support, encouragement, and helpful critiques throughout the project development process.

Lastly, we are deeply grateful to our families for their unwavering encouragement and motivation during this journey, and to everyone who provided guidance and technical assistance. This project has been a valuable learning experience that will greatly benefit us in our future careers.

Abstract

The Scooter Rental Kandy system is a web-based application developed to improve the management and accessibility of scooter rental services for tourists and local travellers in Kandy. The main purpose of this project is to provide an efficient platform where users can easily browse available scooters, make reservations, complete online payments, and track rented vehicles using GPS technology. In traditional rental systems, many operations are handled manually, which can lead to delays, booking conflicts, payment difficulties, and challenges in monitoring vehicle usage. To overcome these issues, this system introduces a digital solution that simplifies both customer interactions and administrative tasks. Customers are able to view scooter details, check availability, select rental periods, make secure online payments, and use GPS tracking for navigation and safety purposes. At the same time, administrators can manage scooters, bookings, customer records, payments, and rental activities through a centralized system. The system also includes additional features such as booking confirmations, notifications, rental history, and future support for advanced analytics and promotional offers. These features help improve the overall efficiency, transparency, and user experience of the platform. This report discusses the background, requirement analysis, system design, and implementation of the Scooter Rental Kandy application. Overall, the system aims to provide a more organized, reliable, and user-friendly approach to managing scooter rental services in Kandy.

Table of Contents

Contents

Abstract iv
List of Figures xi
List of Tables xii
CHAPTER 1 INTRODUCTION xiii
1.1. Client Details xiii
1.2. Problem xiii
1.3. Proposed Solution xiv
1.4. Target Users And Unique features xiv
Included Features xiv
1.5. Client Agreement Letter xv
Chapter 2: Software Requirements Specification xvi
Software Requirements Specification xvi
2.1 Definitions, Acronyms, and Abbreviations xvi
2.2 Introduction 1
2.3 Purpose 2
2.4 Scope 3
2.4.1 Inclusions 3
2.4.2 Exclusions 3
2.5 Document Overview 4
2.6 Project Scope 5
2.6 Stakeholders 6
2.7 Functional Requirements 7
2.8 Non-Functional Requirements 9
2.9 Business Rules 11
2.10 Constraints 12
2.11 Assumptions 13
2.12 Use Case Diagrams 14
2.13: Use Case Descriptions 15
2.13.1: User Registration 15
Basic Flow: 15
Alternate Flow: 15
2.13.2: User Login 15

Basic Flow: 15
Alternate Flow: 15
2.13.3: Search Scooters 16
Basic Flow: 16
Alternate Flow: 16
2.13.4: Book a Scooter 16
Basic Flow: 16
Alternate Flow: 17
2.13.5: Cancel Booking 17
Basic Flow: 17
Alternate Flow: 17
2.13.6: Admin – Add / Update / Delete Scooter 17
Basic Flow: 17
Alternate Flow: 18
2.13.7: Admin – View Reports 18
Basic Flow: 18
Alternate Flow: 18
2.14: Activity Diagrams 19
2.14.1 Login 19
2.14.2 Registration 20
2.14.3 Add Scooter 21
2.14.4 Manage Booking By Admin 22
2.14.5 Booking By User 23
2.15: External Interface Requirements 24
2.15.1 User Interface Requirements 24
2.15.2 Hardware Interface Requirements 24
2.15.3 Software Interface Requirements 25
2.15.4 Communications Interface Requirements 25
2.16: Data Requirements 26
2.16.1 Data Entities Overview 26
2.16.2 User 26
2.16.3 Scooter 27
2.16.4 Booking 27
2.16.5 Payment 28
2.16.6 GPS Log 29
2.16.7 Camping Gear 29
2.16.8 Data Formats and Standards 30

2.16.9 Data Privacy and Compliance 30
2.17: Alternative Solutions Considered 31
2.17.1 Manual Booking System 31
2.17.2 Mobile-Only Application 31
2.17.3 Selected Solution: Web and Mobile Responsive System 31
2.18: Feasibility Study 32
2.18.1 Technical Feasibility 32
2.18.2 Economic Feasibility 32
2.18.3 Operational Feasibility 32
2.18.4 Schedule Feasibility 33
2.19: Risk Analysis 34
2.19.1 Risk Register 34
References 36
Appendix A: Client Agreement 37
A.1 Client Requirement Sign-off 37
A.2 Sample Agreement 37
1. Project Title 37
2. Background and Problem Statement 37
3. Purpose of the Proposed System 38
4. Scope of Collaboration 38
5. Nature of the Project 38
6. Confidentiality 38
7. Intellectual Property 39
8. Term and Termination 39
A.3 Copy of Original Agreement 39
1. Project Title 40
2. Background and Problem Statement 40
3. Purpose of the Proposed System 40
4. Scope of Collaboration 40
5. Nature of the Project 41
6. Confidentiality 41
7. Intellectual Property 41
8. Term and Termination 41
A.4 Official Client Feedback Summary – SRS and SDS Version 1.0 42
1. Summary of Feedback 42
2. Key Stakeholder Observations 42
3. Action Plan in Response to Feedback 43

4. Client Acknowledgement 43
Chapter 3 44
Software Design Specification 44
3.1 Architectural Design 44
3. 1.1 High-Level Architecture 44
3.1.2 Justification for Architecture Choice 46
3.2.Database Design 47
3.2.1 Entity-Relationship (ER) Diagram 47
3.2.2 Entity Descriptions and Attributes 48
User 48
Scooter 49
Booking 50
Payment 51
GPS Log 51
Camping Gear 52
3.2.3 Key Relationships 52
3.2.4 Normalization 53
3.3: Class Diagram 54
3.1 Domain Class Descriptions 54
3.2 Enumerations 55
3.3 Relationships and Multiplicity 56
3.4: Sequence Diagrams 56
3.4.1 User Registration 56
3.4.2 Scooter Booking 58
3.4.3 GPS-Based Billing Calculation 59
3.4.4 Admin Report Generation 61
3.5: UI/UX Design 63
3.5.1 Design Principles 63
3.5.2 Screen Descriptions and Accessibility Considerations 64
3.5.3 User Navigation Flow 64
3.6: Security Design 65
3.6.1 Authentication Mechanism 65
3.6.2 Security Design Summary 66
3.6.3 Input Validation and Sanitization Strategy 67
3.6.4 Encryption Standards 67
3.7: Deployment Design 68
3.7.1 Hosting Plan 68

3.7.2 Environment Specification 69
3.7.3 Network Diagram 70
3.7.4 CI/CD Pipeline Overview 71
3.8: Data Requirements 72
3.8.1 Overview 72
3.8.2 Data Entities and Attributes 72
3.8.2.1 User Data 72
3.8.2.2 Scooter Data 73
3.8.2.3 Booking Data 73
3.8.2.4 Payment Data 74
8.2.5 GPS Log Data 75
3.8.2.6 Camping Gear Data 75
3.8.3 Data Volume and Retention 76
3.8.4 Data Formats and Standards 76
3.8.5 Data Privacy and Compliance 77
References 78
Chapter 4 79
Prototype Development & Client Validation 79
4.1 UI/UX Design 79
4.1.1 Design Principles 79
4.1.2 Key Screen Wireframes 79
5.2 Screen Descriptions and Accessibility Considerations 79
4.1.3 Prototype Navigation Flow 85
4.2 Client Involvement Section 86
4.2.1 Client Demonstration Session 86
4.2.2 Client Feedback Report 86
4.2.3 Official Client Feedback Proof 87
4.2.3 Official Client Feedback – SRS and SDS Review 88
4.3 Ethical Considerations 89
4.3.1 Data Privacy and Consent 89
4.3.2 Safety and Accountability 89
4.3.3 Accessibility and Inclusivity 89
4.3.4 Academic Integrity 90
4.4 Reflection on Stakeholder Communication 90
4.4.1 Communication Approach 90
4.4.2 Challenges Encountered 90
4.4.3 Lessons Learnt 91

4.5 Formal Sign-off from the Client for the Development 91
Chapter 5 93
Conclusion 93
5.1 Degree of Objectives Met 93
5.2 Usability, Accessibility, Reliability and Friendliness 93
5.3 Limitations and Drawbacks 94
5.4 Timeline 94
5.5 Future Modifications, Improvements and Extensions Possible 94
Appendix B: Project Poster 96
Appendix C: References (IEEE Format) 97

List of Figures

Figure : Client Agreement Letter……ix
Figure 1: Use Case Diagram – Scooter Rent Kandy 14
Figure 2.1: Activity Diagram – Login 19
Figure 2.2: Activity Diagram – Registration 20
Figure 2.3: Activity Diagram – Add Scooter 21
Figure 2.4: Activity Diagram – Manage Booking 22
Figure 2.5: Activity Diagram – Booking the Scooter 23
Figure 3: High-Level System Architecture Diagram 47
Figure 4: Entity-Relationship (ER) Diagram (Chen Notation) 48
Figure 5: UML Class Diagram 52
Figure 6: Sequence Diagram – User Registration 55
Figure 7: Sequence Diagram – Scooter Booking 57
Figure 8: Sequence Diagram – GPS-Based Billing Calculation 58
Figure 9: Sequence Diagram – Admin Report Generation 59
Figure 10: UI Navigation Flow Diagram 62
Figure 11: Deployment Network Diagram 67
Figure B.1: Project Poster – Scooter Rent Kandy 88

List of Tables

Table 1: Definitions, Acronyms, and Abbreviations x
Table 2: Functional Requirements (FR-001 to FR-027) 7
Table 3: Non-Functional Requirements (NFR-001 to NFR-017) 9
Table 4: Business Rules (BR-001 to BR-010) 11
Table 5: User Data Entity 26
Table 6: Scooter Data Entity 27
Table 7: Booking Data Entity 27
Table 8: Payment Data Entity 28
Table 9: GPS Log Data Entity 29
Table 10: Camping Gear Data Entity 29
Table 11: Risk Register (R-001 to R-009) 34
Table 12: IEEE References 36
Table 13: Justification for Architecture Choice 47
Table 14: Architectural Layers and Responsibilities 47
Table 15: Normalization Analysis (1NF – BCNF) 51
Table 16: Domain Class Descriptions 53
Table 17: Enumerations 54
Table 18: Sequence Steps – User Registration 55
Table 19: Sequence Steps – Scooter Booking 57
Table 20: Sequence Steps – GPS-Based Billing Calculation 58
Table 21: Sequence Steps – Admin Report Generation 59
Table 22: Screen Descriptions and Accessibility Considerations 61
Table 23: Security Design Summary 63
Table 24: Encryption Standards 64
Table 25: AWS Deployment Components 65
Table 26: Environment Specification 66
Table 27: Key Screen Wireframes and UI Elements 72
Table 28: Official Client Feedback – Metadata 75
Table 29: Client Feedback Action Plan 76
Table 30: Client Sign-off Details 79
Table 31: Original Agreement – Student Team 85
Table 32: Official Client Feedback Summary – Metadata 86
Table 33: Official Client Feedback Summary – Acknowledgement 87
Table 34: References (IEEE Format) 89

CHAPTER 1 INTRODUCTION

1.1. Client Details

The client for this project is Scooter Rental Kandy, a business operating in Kandy that specializes in providing scooter rental services for tourists and local travellers. The company supports customers who require convenient, affordable, and flexible transportation options to explore the city and surrounding attractions. Currently, many of the business operations such as booking management, customer record handling, payment tracking, and vehicle availability updates are managed manually or through partially digital methods, which reduces efficiency and creates challenges in daily operations.

The client has identified the need for a centralized and automated system to streamline rental operations, improve customer data management, monitor scooter availability, and enhance overall service quality. This project is carried out in collaboration with the client to analyse requirements, design, and develop a software solution that effectively addresses these operational needs.

1.2. Problem

The Scooter Rental application solves key problems faced in present-day vehicle rental services, including the difficulty of manually locating available scooters, managing customer bookings, and handling payments efficiently. Existing rental systems often lack essential features such as real-time scooter allocation, GPS tracking to identify the nearest available scooter for customers, secure online payment methods, and automated booking management. These limitations can lead to delays, poor customer experience, booking conflicts, and inefficient use of available scooters.

The Scooter Rental system aims to improve these operations by providing an automated and centralized platform that enables users to book scooters online, locate available scooters based on distance using GPS tracking, make secure online payments, and receive a faster and more convenient rental experience. This ensures better service efficiency, accurate scooter management, and higher customer satisfaction.

1.3. Proposed Solution

The Scooter Rental application has been developed with the objective of creating a modern mobile platform that allows tourists and local travelers to easily rent scooters and travel equipment in a fast, secure, and convenient manner. The system improves customer satisfaction and business efficiency by offering online booking, GPS-based scooter tracking to locate the nearest available vehicle, secure online payment facilities, and access to additional rental services such as tents and outdoor equipment.

The main objectives of the system are to provide an online booking facility for scooters and rental equipment, enable real-time GPS tracking to identify available scooters based on customer location, and integrate a secure digital payment method for hassle-free transactions. In addition, the application aims to maintain accurate availability records, provide ratings and feedback features to improve service quality, offer equipment rental options such as tents, and support business growth through a centralized and user-friendly digital platform.

1.4. Target Users And Unique features

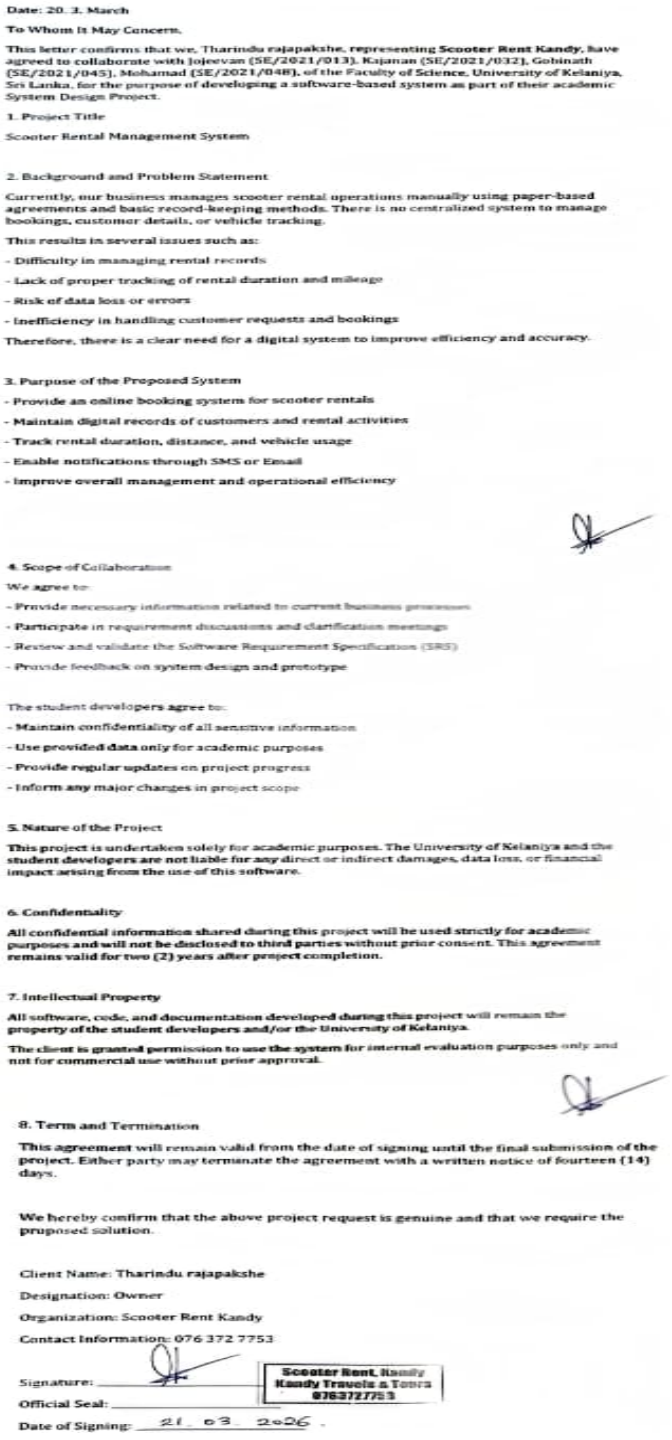
The Scooter Rental application is an important tool in improving transportation services for local users and travelers by making it easy to rent scooters and travel equipment through a digital platform. It enhances convenience, accessibility, and customer satisfaction by simplifying the booking process, providing real-time scooter availability, and ensuring secure transactions. The scope of the application covers several key features designed to improve the rental experience while supporting efficient business operations.

Included Features

- Online Scooter Booking: Users can reserve scooters quickly through the application without manual inquiries or physical visits.
- GPS Tracking and Locator: Users are able to locate nearby available scooters based on their current location, helping them save time and access transport easily.
- Real-Time Availability Management: Administrators can update scooter and equipment availability instantly so users can view accurate booking information.
- Secure Payment System: The application supports safe online payments, allowing users to complete transactions conveniently without cash handling.
- Equipment Rental Services: Users can also rent travel items such as tents and other outdoor equipment through the same platform.
- Ratings and Reviews: Customers can provide feedback and ratings on scooters and services, helping maintain quality and build trust for future users.
- User Profiles and Booking History: Customers can manage their accounts, view previous bookings, and track rental details for future use.

1.5. Client Agreement Letter

The client has reviewed and approved the proposed Scooter Rental system plan, including its objectives, features, and overall project scope. They have confirmed that the identified problem statement accurately reflects their current operational challenges and have expressed willingness to collaborate throughout the development process. The client is satisfied with the proposed solution and has granted the necessary approvals to proceed with the project. An officially signed document confirming this agreement is attached below.



Date: 20. 3. March

To Whom It May Concern,

This letter confirms that we, Tharindu rajapakshe, representing Scooter Rent Kandy, have agreed to collaborate with Jojeevan (SE/2021/013), Kajanan (SE/2021/032), Gobinath (SE/2021/045), Mohamad (SE/2021/048), of the Faculty of Science, University of Kelaniya, Sri Lanka, for the purpose of developing a software-based system as part of their academic System Design Project.

1. Project Title

Scooter Rental Management System

2. Background and Problem Statement

Currently, our business manages scooter rental operations manually using paper-based agreements and basic record-keeping methods. There is no centralized system to manage bookings, customer details, or vehicle tracking.

This results in several issues such as:

- Difficulty in managing rental records
- Lack of proper tracking of rental duration and mileage
- Risk of data loss or errors
- Inefficiency in handling customer requests and bookings

Therefore, there is a clear need for a digital system to improve efficiency and accuracy.

3. Purpose of the Proposed System

- Provide an online booking system for scooter rentals
- Maintain digital records of customers and rental activities
- Track rental duration, distance, and vehicle usage
- Enable notifications through SMS or Email
- Improve overall management and operational efficiency

4. Scope of Collaboration

We agree to:

- Provide necessary information related to current business processes
- Participate in requirement discussions and clarification meetings
- Review and validate the Software Requirement Specification (SRS)
- Provide feedback on system design and prototype

The student developers agree to:

- Maintain confidentiality of all sensitive information
- Use provided data only for academic purposes
- Provide regular updates on project progress
- Inform any major changes in project scope

5. Nature of the Project

This project is undertaken solely for academic purposes. The University of Kelaniya and the student developers are not liable for any direct or indirect damages, data loss, or financial impact arising from the use of this software.

6. Confidentiality

All confidential information shared during this project will be used strictly for academic purposes and will not be disclosed to third parties without prior consent. This agreement remains valid for two (2) years after project completion.

7. Intellectual Property

All software, code, and documentation developed during this project will remain the property of the student developers and/or the University of Kelaniya.

The client is granted permission to use the system for internal evaluation purposes only and not for commercial use without prior approval.

8. Term and Termination

This agreement will remain valid from the date of signing until the final submission of the project. Either party may terminate the agreement with a written notice of fourteen (14) days.

We hereby confirm that the above project request is genuine and that we require the proposed solution.

Client Name: Tharindu rajapakshe

Designation: Owner

Organization: Scooter Rent Kandy

Contact Information: 076 372 7753

Signature: _____

Official Seal: _____

Scooter Rent, Kandy
Kandy Travels & Tours
0763727753

Date of Signing: 21 03 2026

Chapter 2: Software Requirements Specification

Software Requirements Specification

2.1 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition / Description
SRS	Software Requirements Specification – a document describing all functional and non-functional requirements of the system.
FR	Functional Requirement – a requirement describing a function or feature the system must perform.
NFR	Non-Functional Requirement – a requirement describing how the system performs, rather than what it does.
UI	User Interface – the part of the system that allows interaction with users.
API	Application Programming Interface – a set of protocols and tools that allow software applications to communicate.
DB	Database – an organized collection of data stored and accessed electronically.
GPS	Global Positioning System – a satellite-based navigation and location tracking system.
KM	Kilometers – unit of distance used for distance-based rental charges.
RBAC	Role-Based Access Control – a method of restricting system access based on user roles.
JWT	JSON Web Token – an open standard for securely transmitting information as a JSON object.
AI	Artificial Intelligence – the simulation of human intelligence by computer systems.
AWS	Amazon Web Services – cloud platform used for deployment, server, and storage solutions.
PCI DSS	Payment Card Industry Data Security Standard – a security standard for organizations handling card payments.
OTP	One-Time Password – a password valid for a single login session or transaction.
QR	Quick Response – a machine-readable barcode used for check-in/check-out of vehicles.

2.2 Introduction

Scooter Rent Kandy is a scooter rental management system developed to make the rental process simple, fast, and fully digital. Traditional rental systems are mostly manual, which can lead to errors, delays, and difficulty in tracking vehicles. This system addresses those problems by using modern technologies such as GPS tracking and the Google Maps API. With these features, the system is able to track the real-time location of scooters, calculate the distance travelled, and automatically generate billing based on rental time and distance.

The application is developed using Spring Boot for the backend and React for the frontend, which ensures a smooth, scalable, and responsive system. It is designed to handle multiple users efficiently while maintaining performance and security.

The system provides two main interfaces: a User Panel and an Admin Panel. In the User Panel, customers are able to browse available scooters, book rentals, accept digital contracts, and make secure online payments. They may also view their booking history and receive notifications via SMS or email. In the Admin Panel, administrators are able to manage scooters, monitor bookings, track mileage and GPS data, detect speed violations, and generate reports for business analysis.

Role-Based Access Control (RBAC) is implemented to ensure that only authorized users are able to access specific features. This document describes the system requirements, features, and overall functionality to assist developers and stakeholders in clearly understanding the system.

2.3 Purpose

The purpose of this document is to clearly specify the requirements of the Scooter Rent Kandy system. It describes what the system shall do, how it shall operate, and what features must be included. This document is used as a guide to ensure that the system is developed correctly in accordance with the client's needs.

This document assists all parties involved in the project to understand the system in a consistent manner. It explains the functions, system behavior, and important rules that must be followed during development. By maintaining this document, confusion and misunderstandings are reduced, and the development process becomes more organized and efficient.

This document is primarily created for developers, testers, project supervisors, and stakeholders. Developers will use this document to design and build the system correctly. Testers will use it to verify that the system operates as expected and meets all specified requirements. Project supervisors and evaluators will use it to review the system and confirm that it adheres to the required standards. Stakeholders and the client may also refer to this document to understand how the system will operate and to confirm that their requirements are included.

In addition, this document may be used in the future for system maintenance and improvements. If any changes are required, this document will serve as a reference to the original system design. Overall, this document ensures that the Scooter Rent Kandy system is developed in a clear, correct, and well-organized manner, meeting all user and client expectations.

2.4 Scope

The system name is Scooter Rent Kandy, which is a scooter rental management system designed to handle the full rental process in a digital manner. The system allows users to search, book, and rent scooters conveniently through an online platform. It also enables administrators to manage the entire rental operation efficiently.

The system will provide features including user registration and login, scooter browsing, booking management, and secure online payments. GPS tracking will be used to monitor the real-time location of scooters, and the Google Maps API will be integrated to calculate travel distance and routes. Based on this data, the system will automatically generate rental charges. Users will also be able to view their booking history and receive notifications.

For administrators, the system will include features such as adding and managing scooters, tracking vehicle usage, monitoring speed violations, and generating operational reports. Role-Based Access Control (RBAC) will be used to restrict access to different system functions.

2.4.1 Inclusions

- Online user registration, login, and profile management.
- Scooter browsing, search, and real-time availability display.
- Online booking with GPS-based pickup and drop-off selection.
- Secure online payment processing (card and gateway).
- Digital contract acceptance prior to rental.
- Automated billing based on time and GPS distance.
- Admin dashboard for fleet management, booking monitoring, and reporting.
- SMS and email notifications for booking confirmations and alerts.

2.4.2 Exclusions

- Physical maintenance or mechanical repair management of scooters is not included.
- Fuel management and insurance processing are excluded from the system scope.
- Offline bookings are not supported; all operations must be performed through the online platform.

- Third-party delivery services and AI-based recommendations are not included in the current version.

2.5 Document Overview

This document is organized to provide a comprehensive view of the Scooter Rent Kandy system, covering all requirements, diagrams, and references in a structured manner.

Chapters 1 through 3 introduce the project and explain its purpose, intended audience, and system scope. Chapter 4 (this chapter) describes the organization of the document. The Definitions, Acronyms, and Abbreviations section in the front matter defines all domain-specific terms used throughout the document.

Chapters 5 and 6 define the project scope boundaries and identify all stakeholders who will use or be affected by the system.

Chapter 7 presents the functional requirements in a tabular format with unique identifiers, priority levels, source stakeholders, and approval status. Chapter 8 presents the non-functional requirements, categorized according to ISO/IEC 25010 quality attributes, each with a measurable acceptance criterion.

Chapters 9 through 11 document the business rules, constraints, and assumptions that govern the project. Chapters 12 through 14 provide visual representations including use case diagrams, use case descriptions, and activity diagrams.

Chapter 15 specifies external interface requirements. Chapter 16 lists data requirements. Chapter 17 presents alternative solutions that were considered prior to the current design. Chapter 18 provides a feasibility study. Chapter 19 presents the risk analysis with a formal risk register. The References section lists all cited sources in IEEE format. The Agreement appendix contains the client sign-off documentation.

2.6 Project Scope

The Scooter Rent Kandy system is designed to make the process of renting and returning scooters easier and faster for both users and administrators. The system will allow users to search for available scooters, make reservations, and complete payments online. Administrators will be able to manage the scooter fleet, track active rentals, update vehicle status, and generate operational reports.

The system will include features such as user registration and login, rental booking history, and status notifications. It will also provide an administrative dashboard for monitoring all activities, including scooter availability, GPS tracking, speed violation alerts, and rental statistics.

The project scope encompasses both the web platform and mobile browser access, along with integration with third-party services including GPS tracking, payment gateways, and mapping APIs. Data security for users' personal and payment information is a primary concern throughout the scope.

Explicitly excluded from the scope are physical scooter maintenance management, insurance claim processing, offline operations, and unrelated transport services. The system focuses solely on scooter rental operations, user management, and system reporting.

2.6 Stakeholders

Stakeholders are all individuals, groups, or organizations who have an interest in the Scooter Rent Kandy system or are affected by its operation. Identifying stakeholders ensures that the system meets the needs of all parties involved.

Users (Renters): The primary stakeholders are customers who rent scooters for personal or leisure use. Users interact with the system to browse available scooters, make bookings, process payments, and view rental history. Their feedback and requirements are essential to ensuring the system is easy to use, reliable, and secure.

Administrators: Administrators manage the overall system, including the scooter fleet, user accounts, booking oversight, and reporting. They ensure the system operates smoothly, handle maintenance updates, and monitor rental statistics. Administrators also manage security compliance and system integrity.

Maintenance Staff: Maintenance staff require access to information about scooter availability, service schedules, and repair notifications. The system assists them in planning repairs, tracking vehicle conditions, and maintaining operational efficiency.

Payment Service Providers: External stakeholders such as Stripe and PayPal are integrated with the system to handle secure online transactions. Ensuring accurate and safe payment processing is critical to the credibility of the system.

Regulatory Authorities: Government and legal entities may be considered stakeholders to ensure that the system complies with local laws, data protection regulations, and vehicle rental safety standards.

2.7 Functional Requirements

Table 2: Functional Requirements

ID	Description	Priority	Source	Status
FR-001	The system shall allow users to register by providing personal details (name, email, phone number, and government-issued ID).	High	User	Approved
FR-002	The system shall allow registered users to log in securely using their email address and password.	High	User	Approved
FR-003	The system shall display available scooters with details such as model, price per hour, and availability status.	High	User	Approved
FR-004	The system shall allow users to search for scooters based on location, type, and price range.	Medium	User	Approved
FR-005	The system shall allow users to book a scooter for a selected date and time period.	High	User	Approved
FR-006	The system shall allow users to make secure online payments for bookings.	High	User	Approved
FR-007	The system shall allow users to view their booking history.	Medium	User	Approved
FR-008	The system shall allow users to cancel a booking before the scheduled pickup time, subject to the cancellation policy.	Medium	User	Approved
FR-009	The system shall allow administrators to log in securely via a dedicated admin interface.	High	Admin	Approved
FR-010	The system shall allow administrators to add, update, and remove scooters from the fleet.	High	Admin	Approved
FR-011	The system shall allow administrators to manage user accounts (view, edit, deactivate).	Medium	Admin	Approved
FR-012	The system shall allow administrators to generate reports on bookings, revenue, and payments.	Low	Admin	Approved
FR-013	The system shall allow users to update their profile information.	Medium	User	Approved
FR-014	The system shall allow users to reset their password via a verified email link.	High	User	Approved
FR-015	The system shall send booking confirmation notifications to users via SMS and email.	High	System	Approved
FR-016	The system shall display real-time scooter availability status.	High	User	Approved
FR-017	The system shall allow users to select pickup and drop-off locations using a map interface.	High	User	Approved

FR-018	The system shall calculate the total rental cost based on rental duration and distance travelled.	High	System	Approved
FR-019	The system shall support multiple payment methods (credit/debit card, online payment gateway).	Medium	User	Approved
FR-020	The system shall generate and send an invoice to the user after successful payment.	Medium	System	Approved
FR-021	The system shall allow administrators to approve or reject new scooter listings before they become available.	Medium	Admin	Approved
FR-022	The system shall allow administrators to view all active bookings in real time.	High	Admin	Approved
FR-023	The system shall allow administrators to track scooter usage, mileage, and availability.	Medium	Admin	Approved
FR-024	The system shall allow administrators to block or deactivate user accounts.	High	Admin	Approved
FR-025	The system shall maintain a complete and auditable record of all transactions.	High	System	Approved
FR-026	The system shall allow users to submit ratings and reviews for scooters after a completed rental.	Low	User	Approved
FR-027	The system shall allow users to contact customer support through the system.	Low	User	Approved

2.8 Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Category	Requirement / Description	Measurable Acceptance Criterion
NFR-001	**Performance**	The system shall load the homepage within an acceptable response time.	95% of homepage requests shall load within 3 seconds under normal operating conditions.
NFR-002	**Performance**	The system shall return scooter search results promptly.	95% of search queries shall respond within 2 seconds under normal load.
NFR-003	**Reliability**	The system shall maintain high availability for all users.	The system shall achieve 99% uptime, measured on a monthly basis.
NFR-004	**Reliability**	The system shall perform automated data backups.	Daily backups shall complete successfully without errors 100% of the time; data shall be recoverable within 1 hour.
NFR-005	**Security**	All user passwords shall be stored securely.	Passwords shall be hashed and salted using BCrypt (cost factor ≥ 12); plain-text storage is strictly prohibited.
NFR-006	**Security**	All payment transactions shall be processed securely.	100% of transactions shall comply with PCI DSS standards.
NFR-007	**Usability**	The system shall provide a user-friendly and intuitive interface.	90% of test participants shall be able to complete a booking without external assistance.
NFR-008	**Usability**	The system shall be compatible with multiple devices and browsers.	The application shall render correctly on the top 5 desktop browsers and top 3 mobile browsers.
NFR-009	**Scalability**	The system shall handle increasing numbers of concurrent users.	The system shall support at least 1,000 concurrent users without performance degradation.
NFR-010	**Maintainability**	The system shall be easy to update and maintain.	System updates and patches shall be deployable within 30 minutes with zero downtime.
NFR-011	**Portability**	The system shall be compatible with major operating systems.	The system shall function correctly on Windows, macOS, Android, and iOS platforms.
NFR-012	**Performance**	Notifications and alerts shall be delivered promptly.	95% of SMS and email notifications shall be delivered within 1 minute of the triggering event.
NFR-013	**Security**	The system shall maintain audit logs of all user activities.	All log entries shall be timestamped, attributed to a user,

			and stored securely for a minimum of 1 year.
NFR-014	**Reliability**	The system shall archive historical booking records automatically.	All bookings older than 1 year shall be archived automatically on a daily schedule.
NFR-015	**Usability**	The system shall support multilingual users.	Users shall be able to switch the interface language between English, Sinhala, and Tamil from any page.

2.9 Business Rules

Business rules define the operational policies and constraints under which the Scooter Rent Kandy system must operate. These rules are derived from the client's business requirements and must be enforced by the system at all times.

Table 4: Business Rules

ID	Business Rule
BR-001	Users must be at least 18 years of age to rent a scooter.
BR-002	Each user may book only one scooter at a time.
BR-003	Scooters must be returned to the scheduled drop-off location by the agreed return time.
BR-004	Payment must be completed and confirmed before a booking is activated.
BR-005	Administrators must approve any new scooter added to the fleet before it becomes available for booking.
BR-006	Users may not cancel a booking within 1 hour of the scheduled pickup time.
BR-007	Late returns shall incur additional charges calculated based on the extra rental duration.
BR-008	Scooters that are under maintenance shall not be available for booking until cleared by an administrator.
BR-009	Users must provide a valid, government-issued identification document during registration.
BR-010	All system transactions must comply with applicable security and data privacy policies.

2.10 Constraints

- The system must operate on web and mobile platforms using responsive design.
- The database must be capable of handling up to 1,000 concurrent users.
- The system must comply with PCI DSS standards for all online payment processing.
- System development must be completed within a period of six months.
- All user data must be stored in compliance with applicable data protection regulations.
- The system must function correctly on standard browsers: Chrome, Firefox, Edge, and Safari.
- Automated backup and recovery processes must complete within one hour each day.
- All APIs must be compatible with third-party mapping and payment services.
- The maximum booking period per scooter is seven days.
- The system interface shall support only the English, Sinhala, and Tamil languages.

2.11 Assumptions

- Users are assumed to have access to a reliable internet connection when using the system.
- Users are assumed to possess a basic level of digital literacy sufficient to navigate a web-based application.
- All scooters in the fleet are assumed to be equipped with functioning GPS devices.
- The client is assumed to provide timely feedback and approvals during the development process.
- Third-party services such as the Google Maps API, Stripe, and AWS are assumed to maintain their advertised service levels and API compatibility.
- Users are assumed to be located in or travelling within the Kandy region of Sri Lanka.
- The hosting infrastructure is assumed to provide a minimum of 99% uptime as per the selected cloud provider’s service level agreement.

2.12 Use Case Diagrams

Figure 1 presents the use case diagram for the Scooter Rent Kandy system, illustrating the interactions between the primary actors (User and Admin) and the system's functional use cases.



Figure 1: Use Case Diagram – Scooter Rent Kandy

2.13: Use Case Descriptions

2.13.1: User Registration

- Actor: User
- Preconditions: The user has access to the internet and a compatible browser.
- Trigger: The user wishes to create a new account.

Basic Flow:

1. The user navigates to the registration page.
2. The user enters personal details (name, email address, phone number, password, and NIC number).
3. The system validates the input for completeness and format.
4. The system stores the user details and confirms successful registration.

Alternate Flow:

- If the user enters invalid or incomplete details, the system displays an error message and prompts re-entry.
- Postconditions: A user account is created and the user is able to log in.

2.13.2: User Login

- Actor: User
- Preconditions: The user has a registered account.
- Trigger: The user wishes to access the system.

Basic Flow:

1. The user navigates to the login page.
2. The user enters their email address and password.
3. The system authenticates the provided credentials.
4. The system grants access to the user dashboard.

Alternate Flow:

- If the credentials are incorrect, the system displays an error message and denies access.
- Postconditions: The user is authenticated and logged into the system.

2.13.3: Search Scooters

- Actor: User
- Preconditions: The user is logged in.
- Trigger: The user wishes to find available scooters.

Basic Flow:

1. The user enters search criteria (location, scooter type, price range).
2. The system retrieves available scooters matching the specified criteria.
3. The system displays the filtered results to the user.

Alternate Flow:

- If no scooters match the search criteria, the system displays a “No results found” message.
- Postconditions: The user is presented with a list of available scooters.

2.13.4: Book a Scooter

- Actor: User
- Preconditions: The user is logged in and a scooter is available.
- Trigger: The user wishes to reserve a scooter.

Basic Flow:

1. The user selects a scooter and specifies the rental period.
2. The system verifies the scooter’s availability for the requested period.
3. The user reviews and accepts the digital rental contract.
4. The user provides payment information and confirms the booking.
5. The system confirms the booking and sends a notification to the user.

Alternate Flow:

- If the selected scooter is unavailable, the system suggests alternative scooters.
- If the payment fails, the booking is cancelled and the user is notified.
- Postconditions: The booking is recorded in the system and a confirmation is sent to the user.

2.13.5: Cancel Booking

- Actor: User
- Preconditions: The user has an active booking.
- Trigger: The user wishes to cancel an existing booking.

Basic Flow:

1. The user selects the booking to be cancelled from their booking history.
2. The system verifies that the cancellation request is within the permitted cancellation window.
3. The system cancels the booking and updates the scooter's availability status.
4. The system notifies the user of successful cancellation.

Alternate Flow:

- If the cancellation window has passed (within 1 hour of pickup), the system rejects the request and informs the user.
- Postconditions: The booking is cancelled and the scooter becomes available for other users.

2.13.6: Admin – Add / Update / Delete Scooter

- Actor: Administrator
- Preconditions: The administrator is logged in.
- Trigger: The administrator wishes to manage the scooter fleet.

Basic Flow:

1. The administrator selects the desired action (add, update, or delete).

2. The administrator provides or updates scooter details (model, registration number, price, availability).
3. The system validates the input and performs the requested action.
4. The system confirms the change to the administrator.

Alternate Flow:

- If the provided details are invalid, the system prompts the administrator to correct the input.
- Postconditions: The scooter records are updated in the system.

2.13.7: Admin – View Reports

- Actor: Administrator
- Preconditions: The administrator is logged in.
- Trigger: The administrator wishes to view operational reports.

Basic Flow:

1. The administrator selects the report type (bookings, payments, or active scooters).
2. The administrator specifies the desired date range.
3. The system generates and displays the requested report.
4. The administrator may choose to export the report in PDF or CSV format.

Alternate Flow:

- If no data is available for the selected period, the system displays a “No data available” message.
- Postconditions: The administrator receives the requested report.

2.14: Activity Diagrams

2.14.1 Login

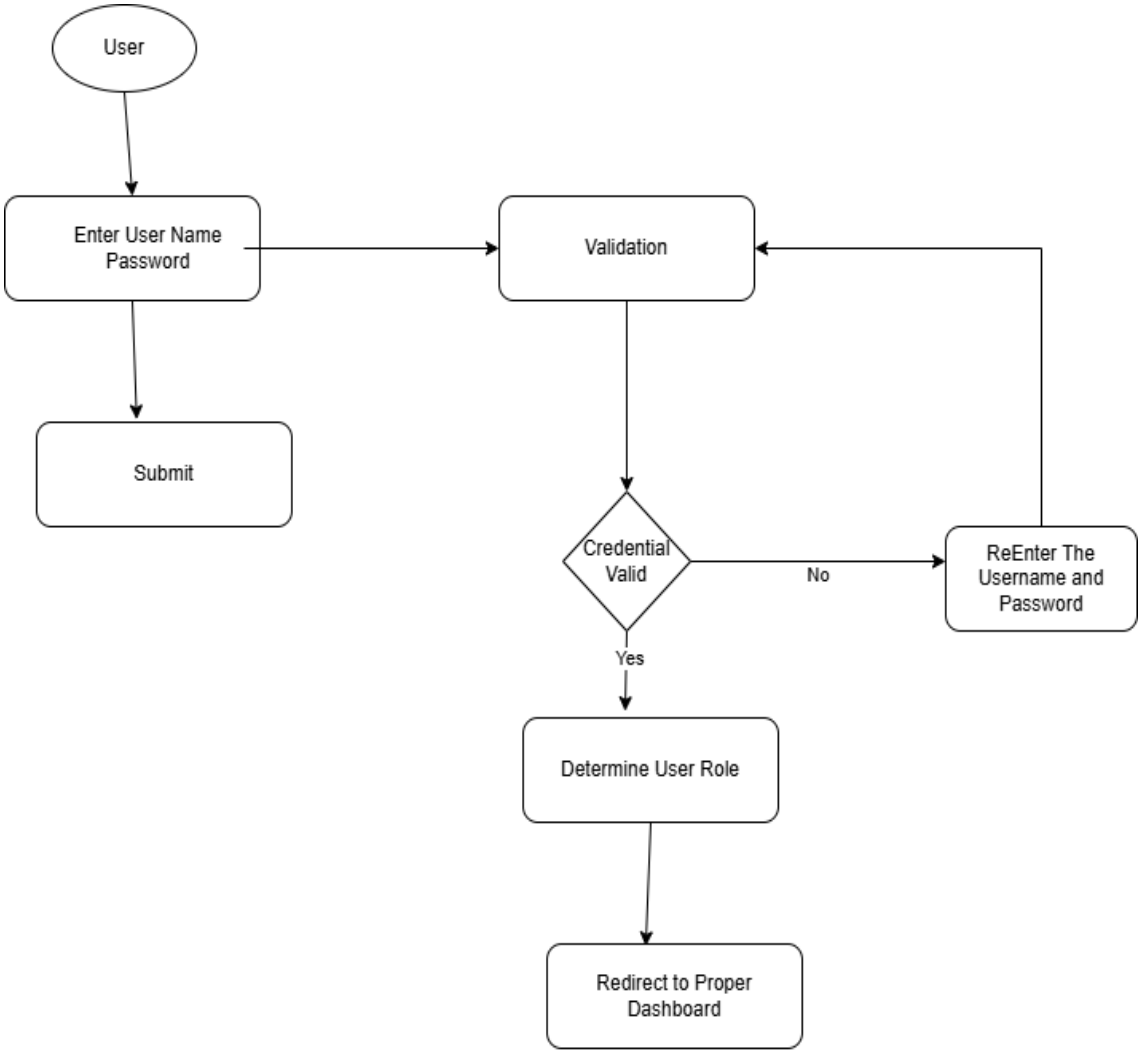


Figure 2.1: Activity Diagram – Login

2.14.2 Registration

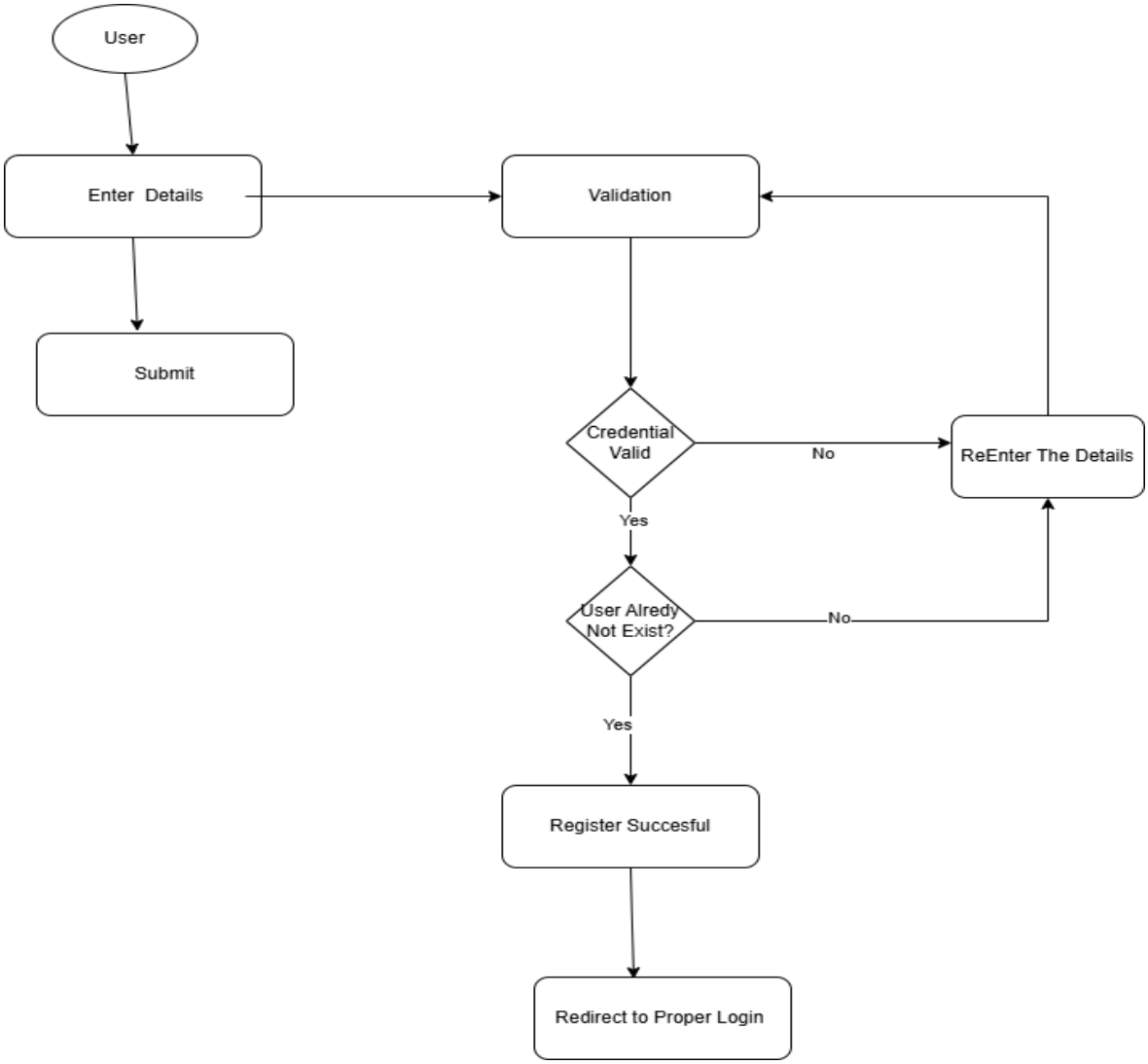


Figure 2.2: Activity Diagram – Registration

2.14.3 Add Scooter

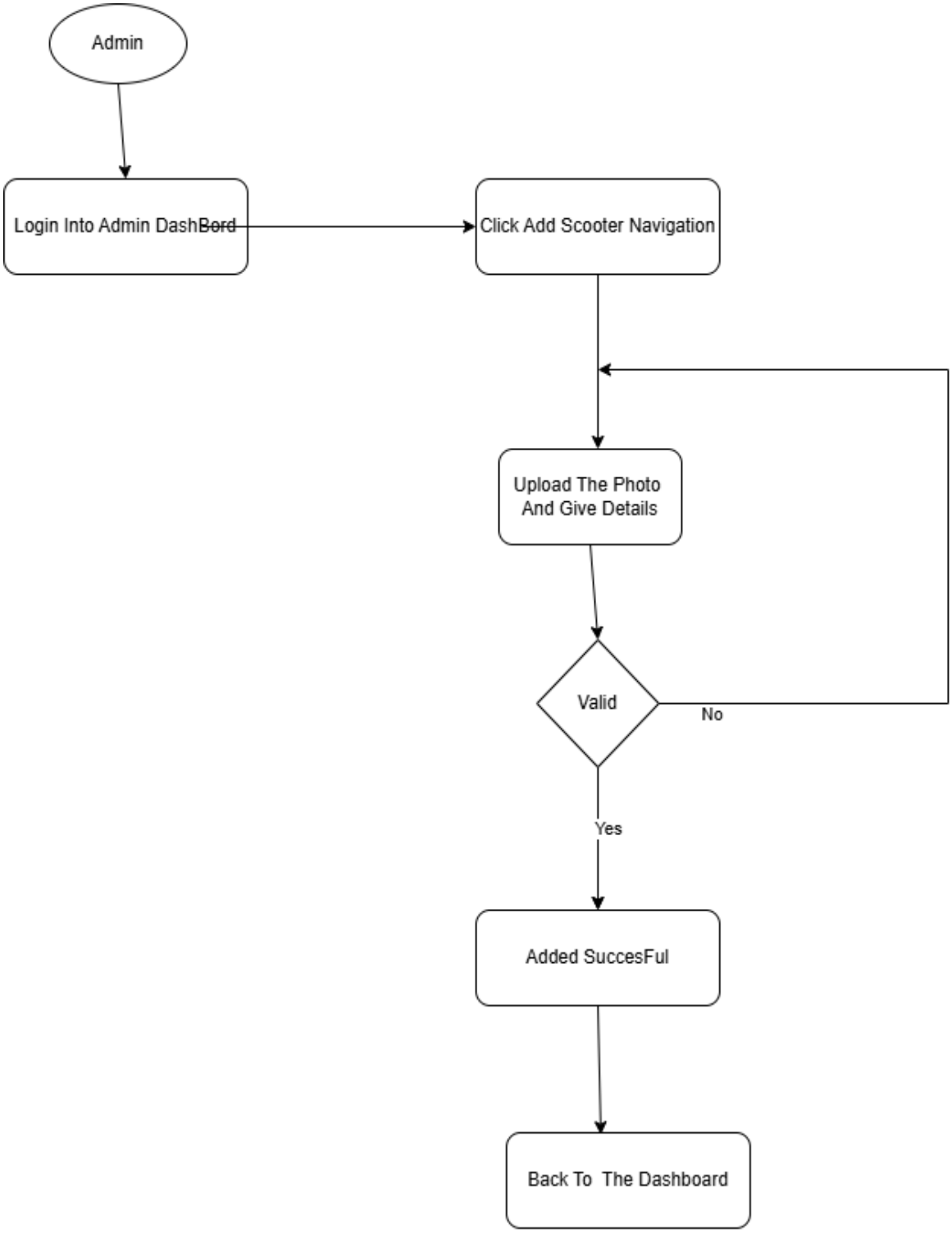


Figure 2.3: Activity Diagram – Add Scooter

2.14.4 Manage Booking By Admin

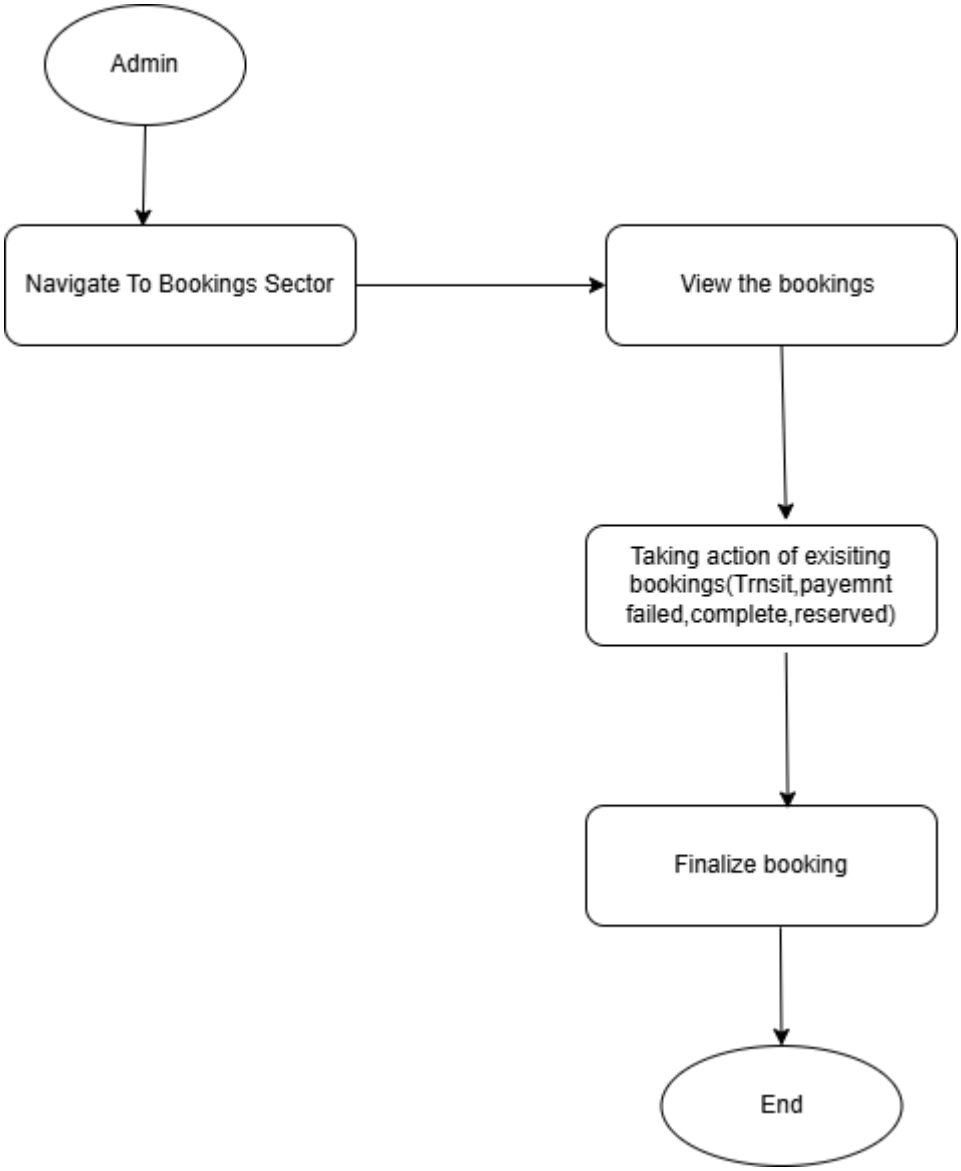


Figure 2.4: Activity Diagram – Manage Booking

2.14.5 Booking By User

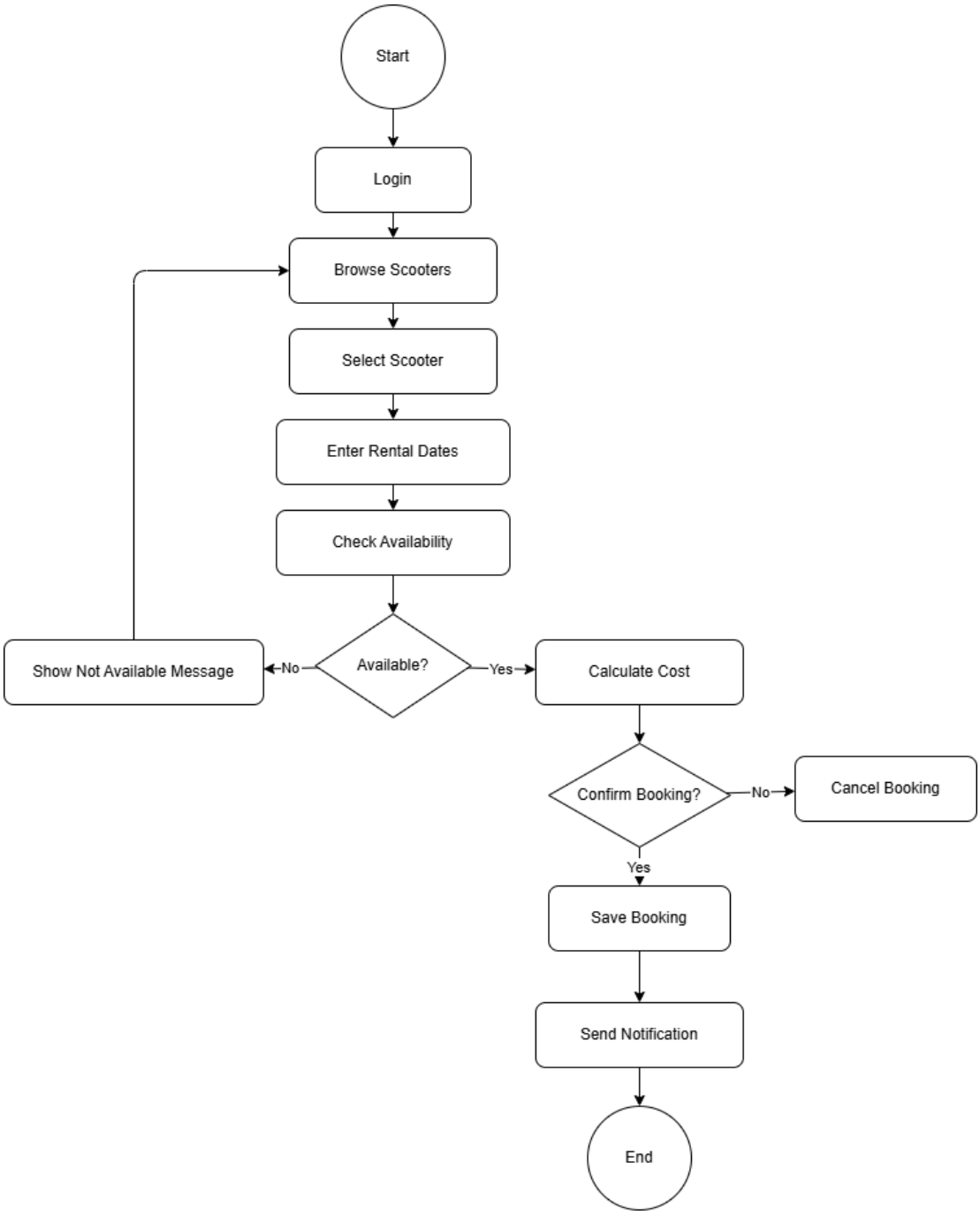


Figure 2.5: Activity Diagram – Booking the Scooter

2.15: External Interface Requirements

2.15.1 User Interface Requirements

- The system must operate on web and mobile platforms using responsive design principles.
- The system shall follow standard navigation patterns (header menus, breadcrumbs, and action buttons) to ensure ease of use.
- The system shall apply a consistent color scheme and typography throughout all pages for visual clarity.
- The system shall provide accessible forms with descriptive labels, input placeholders, and inline validation messages.
- The system shall allow users to view scooter listings with clear images, model descriptions, availability status, and pricing.
- The system shall provide search and filter options based on location, scooter type, and price range.
- The administrator interface shall provide a dashboard with information for managing the scooter fleet, bookings, users, and reports.

2.15.2 Hardware Interface Requirements

- The system shall be accessible via users' desktop computers, laptops, tablets, and mobile devices.
- The system shall be compatible with GPS-enabled mobile devices to support real-time scooter location tracking.
- The system shall support barcode or QR code scanners for scooter check-in and check-out processes where applicable.
- The system shall support on-site payment terminals or card readers for in-person transactions where applicable.
- The system shall be accessible via devices with standard keyboard, mouse, and touchscreen input methods.

2.15.3 Software Interface Requirements

- Integration with payment gateway APIs (e.g., Stripe, PayPal) shall enable secure online payment processing.
- The Google Maps API shall be used for selecting pickup and drop-off locations and for displaying scooter GPS positions.
- Email notification services (e.g., AWS SES, SendGrid) shall handle the delivery of booking confirmations and reminders.
- SMS gateway APIs (e.g., AWS SNS) shall facilitate the delivery of OTPs and booking alerts to users.
- A relational database management system (PostgreSQL or MySQL) shall be used for the storage and retrieval of user, scooter, and booking data.

2.15.4 Communications Interface Requirements

- All client-server communication shall use the HTTPS protocol to ensure secure and encrypted data transfer.
- Interaction between the frontend and backend services shall be via RESTful APIs.
- API request and response payloads shall use the JSON data format for consistency and interoperability.
- TLS 1.2 or higher shall protect all sensitive data exchanged over the network.
- Reliable communication between servers and clients shall use the TCP/IP protocol suite.
- Email notifications shall be transmitted using the SMTP protocol.

2.16: Data Requirements

This chapter specifies the data requirements of the Scooter Rent Kandy system. Each data entity, its key attributes, data types, constraints, and applicable data formats are described below. These requirements are derived from the functional requirements documented in Chapter 7 and inform the database design of the system.

2.16.1 Data Entities Overview

The system manages six primary data entities: User, Scooter, Booking, Payment, GPS Log, and Camping Gear. Each entity is described in detail in the subsections below.

2.16.2 User

The User entity stores information about all registered users of the system, including both customers and administrators.

Attribute	Data Type	Constraint	Description
User ID	UUID	PK, NOT NULL, UNIQUE	Unique system-generated identifier for each user.
First Name	VARCHAR(100)	NOT NULL	The user's given name.
Last Name	VARCHAR(100)	NOT NULL	The user's family name.
Email Address	VARCHAR(255)	NOT NULL, UNIQUE	Used as the login credential; must be in valid email format.
Password (Hashed)	VARCHAR(255)	NOT NULL	BCrypt-hashed password; plain text is never stored.
Phone Number	VARCHAR(20)	NOT NULL	Mobile contact number in international format (e.g., +94771234567).
NIC Number	VARCHAR(20)	NOT NULL, UNIQUE	Sri Lanka National Identity Card number; encrypted at rest.
Role	ENUM	NOT NULL, DEFAULT USER	Determines access level: USER or ADMIN.
Account Status	BOOLEAN	DEFAULT TRUE	TRUE = active; FALSE = blocked by administrator.
Registration Date	TIMESTAMP	NOT NULL	Date and time the account was created (ISO 8601 UTC).

Table 5: User Data Entity

2.16.3 Scooter

The Scooter entity stores details of every scooter in the rental fleet managed by the administrator.

Attribute	Data Type	Constraint	Description
Scooter ID	UUID	PK, NOT NULL, UNIQUE	Unique system-generated identifier for each scooter.
Model Name	VARCHAR(150)	NOT NULL	Scooter model name (e.g., Honda Dio, Yamaha Ray-ZR).
Registration No.	VARCHAR(30)	NOT NULL, UNIQUE	Official vehicle registration plate number.
Availability Status	ENUM	NOT NULL	Current status: AVAILABLE, RENTED, or MAINTENANCE.
Hourly Rate	DECIMAL(10,2)	NOT NULL, > 0	Rental charge per hour in Sri Lankan Rupees (LKR).
Per-KM Rate	DECIMAL(10,2)	NOT NULL, >= 0	Distance-based charge per kilometer in LKR.
Scooter Image URL	VARCHAR(500)	NULLABLE	URL of the scooter image stored in AWS S3.
Total Mileage	DECIMAL(10,2)	DEFAULT 0	Cumulative distance travelled by the scooter in kilometers.
Last Service Date	DATE	NULLABLE	Date of the most recent maintenance service (ISO 8601).

Table 6: Scooter Data Entity

2.16.4 Booking

The Booking entity records every rental transaction initiated by a user, from reservation through to completion or cancellation.

Attribute	Data Type	Constraint	Description
Booking ID	UUID	PK, NOT NULL, UNIQUE	Unique system-generated identifier for the booking.
User ID	UUID	FK → User, NOT NULL	References the user who made the booking.
Scooter ID	UUID	FK → Scooter, NOT NULL	References the scooter reserved for the booking.

Start Time	TIMESTAMP	NOT NULL	Rental start date and time (ISO 8601 UTC).
End Time	TIMESTAMP	NULLABLE	Rental end date and time; NULL if rental is still active.
Pickup Location	VARCHAR(300)	NOT NULL	GPS address of the scooter pickup point.
Drop-off Location	VARCHAR(300)	NULLABLE	GPS address of the return point; NULL until rental ends.
Booking Status	ENUM	NOT NULL	Lifecycle state: PENDING, ACTIVE, COMPLETED, or CANCELLED.
Total Cost	DECIMAL(10,2)	NULLABLE	Calculated final cost in LKR; NULL until booking is completed.
Contract Signed	BOOLEAN	DEFAULT FALSE	TRUE = digital rental contract has been accepted by the user.

Table 7: Booking Data Entity

2.16.5 Payment

The Payment entity records the financial transaction associated with each confirmed booking.

Attribute	Data Type	Constraint	Description
Payment ID	UUID	PK, NOT NULL, UNIQUE	Unique system-generated identifier for the payment record.
Booking ID	UUID	FK → Booking, UNIQUE	References the associated booking (one payment per booking).
Gateway Transaction ID	VARCHAR(255)	NULLABLE	Payment gateway reference number (e.g., Stripe payment intent ID).
Amount	DECIMAL(10,2)	NOT NULL, > 0	Total amount charged to the user in LKR.
Payment Method	ENUM	NOT NULL	Method used: CARD or ONLINE.
Payment Status	ENUM	NOT NULL	Lifecycle state: PENDING, SUCCESS, FAILED, or REFUNDED.
Payment Timestamp	TIMESTAMP	NULLABLE	Date and time the payment was successfully processed (ISO 8601 UTC).

Table 8: Payment Data Entity

2.16.6 GPS Log

The GPS Log entity records real-time location data transmitted by the scooter's GPS device during an active rental. These records are used for distance calculation and speed violation monitoring.

Attribute	Data Type	Constraint	Description
Log ID	UUID	PK, NOT NULL, UNIQUE	Unique identifier for each GPS log entry.
Booking ID	UUID	FK → Booking, NOT NULL	References the active booking this log belongs to.
Latitude	DECIMAL(9,6)	NOT NULL	GPS latitude coordinate in WGS84 decimal degrees.
Longitude	DECIMAL(9,6)	NOT NULL	GPS longitude coordinate in WGS84 decimal degrees.
Speed	DECIMAL(5,2)	NOT NULL, >= 0	Speed of the scooter in km/h at the time of logging.
Recorded At	TIMESTAMP	NOT NULL	Date and time of the GPS log entry (ISO 8601 UTC); logged every 30 seconds.

Table 9: GPS Log Data Entity

2.16.7 Camping Gear

The Camping Gear entity stores details of optional camping equipment (e.g., tents, cookers, lights) that users may add to a scooter booking.

Attribute	Data Type	Constraint	Description
Gear ID	UUID	PK, NOT NULL, UNIQUE	Unique system-generated identifier for the gear item.
Name	VARCHAR(200)	NOT NULL	Name of the camping gear item (e.g., Camping Tent, Gas Cooker, LED Light).
Daily Rate	DECIMAL(10,2)	NOT NULL, > 0	Rental charge per day in LKR.
Stock	INTEGER	NOT NULL, >= 0	Number of units currently available in the inventory.

Table 10: Camping Gear Data Entity

2.16.8 Data Formats and Standards

- All timestamps shall be stored and transmitted in ISO 8601 UTC format (e.g., 2026-04-10T09:00:00Z).
- All monetary values shall be stored as DECIMAL(10,2) representing Sri Lankan Rupees (LKR) to two decimal places.
- GPS coordinates shall be stored as DECIMAL(9,6) following the WGS84 standard.
- All API request and response payloads shall use the JSON data format with camelCase field naming conventions.
- Exported reports shall be available in PDF and CSV formats.
- Scooter images shall be stored in JPEG or PNG format, with a maximum file size of 5 MB per image.

2.16.9 Data Privacy and Compliance

- Personal data (name, email, phone number, NIC number) shall be collected only with explicit user consent at the point of registration.
- NIC numbers shall be encrypted at the application layer using AES-256 before being stored in the database.
- User passwords shall never be stored in plain text; BCrypt hashing (cost factor ≥ 12) shall be applied at all times.
- Payment card data shall never be stored on the system; Stripe tokenization shall be used to ensure PCI DSS compliance.
- Users shall be able to request deletion or export of their personal data in accordance with applicable data protection regulations.
- Audit log entries shall be retained for a minimum of one year.

2.17: Alternative Solutions Considered

2.17.1 Manual Booking System

- A traditional manual system using paper records or telephone-based bookings was considered.
- This approach is simple to implement but is time-consuming and error-prone.
- It does not support real-time availability tracking or systematic booking records.
- This solution was rejected due to its lack of efficiency, scalability, and data reliability.

2.17.2 Mobile-Only Application

- A mobile-only native application was considered to maximize convenience on smartphones.
- This provides a good user experience on mobile but excludes users who prefer desktop or laptop access.
- This solution was rejected because it does not support all user types and device preferences.

2.17.3 Selected Solution: Web and Mobile Responsive System

- A combined web and mobile-responsive system was selected as the most appropriate solution.
- This approach provides accessibility, scalability, and real-time updates across all devices.
- It supports both user and administrator workflows efficiently and is consistent with the client's operational requirements.
- This solution was selected for its flexibility, broad device compatibility, and alignment with the project's technical and business requirements.

2.18: Feasibility Study

2.18.1 Technical Feasibility

The proposed Scooter Rent Kandy system is technically feasible. It is developed using widely available and well-supported technologies, including Java with Spring Boot for the backend, React for the frontend, and a relational database (PostgreSQL). The required hardware, including standard servers and GPS-enabled devices, is readily accessible. Integration with third-party services such as payment gateways and the Google Maps API is achievable through standard RESTful APIs. The development team possesses the necessary technical skills in web development, database management, and API integration. Security measures such as encryption, HTTPS, and role-based access control can be implemented using established frameworks. No major technical barriers have been identified.

2.18.2 Economic Feasibility

The system is economically feasible. The development cost is relatively low, as the project is built using open-source technologies, thereby reducing software licensing expenses. Hardware requirements are minimal and can leverage existing cloud infrastructure. Although recurring costs for cloud hosting, third-party API usage, and maintenance exist, these are manageable within the project budget. In the long term, the system is expected to reduce manual operational costs, increase booking efficiency, and generate additional revenue through improved service capacity. The overall benefits outweigh the estimated costs, making the system economically viable.

2.18.3 Operational Feasibility

The system is operationally feasible. It is designed to be intuitive and straightforward for both customers and administrators. Users are able to register, browse scooters, and complete bookings without technical difficulty. Administrators are able to manage the fleet and generate reports with minimal training. The system reduces manual effort and improves the overall operational workflow. It can be integrated into the client's existing business processes with minimal disruption. Basic training sessions are sufficient for staff to operate the system confidently.

2.18.4 Schedule Feasibility

The system is schedule feasible. Development can be completed within the planned timeframe of six months. The project is divided into clearly defined phases: requirements analysis, system design, development, testing, and deployment. Each phase is allocated appropriate time, with buffer periods to accommodate minor delays. The use of modern development frameworks and collaborative tools supports efficient progress across the team. No significant time-critical risks have been identified that would prevent on-time delivery.

2.19: Risk Analysis

2.19.1 Risk Register

Table 11: Risk Register

Risk ID	Description	Likelihood	Impact	Risk Level	Mitigation Strategy	Category
R-001	Delay in development due to technical challenges	Medium	High	High	Engage experienced developers; maintain a detailed project plan with milestones and buffer time.	Technical
R-002	Changes in requirements during development	High	High	High	Follow a formal requirements management process; require written approval for all scope changes.	Requirements
R-003	System performance degradation under high user load	Medium	High	High	Conduct load testing before deployment; optimise database queries and implement caching.	Technical
R-004	Integration failure with third-party APIs (payment, maps)	Medium	Medium	Medium	Evaluate API reliability prior to selection; implement fallback mechanisms and test integrations early.	Technical
R-005	Data security breaches or unauthorised access	Low	High	Medium	Implement encryption, JWT authentication, RBAC, and regular security audits.	Technical
R-006	Project delay due to limited team resources or availability	Medium	Medium	Medium	Allocate tasks according to team capacity; monitor progress using project management tools.	Schedule / Resource
R-007	Low user adoption due to poor usability	Low	Medium	Low	Conduct usability testing with target users during development; iterate on UI/UX based on feedback.	Requirements
R-008	Hardware or server failure causing service disruption	Low	High	Medium	Use redundant cloud hosting (AWS Multi-AZ); maintain daily backups and a	Technical

					disaster recovery plan.	
R-009	Communication issues within the development team	Medium	Medium	Medium	Hold regular stand-up meetings; use collaboration tools such as Slack and GitHub for coordination.	Resource

References

Table 12: IEEE References

[#]	Reference (IEEE Format)
[1]	IEEE, “IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications,” IEEE, 1998.
[2]	ISO/IEC, “ISO/IEC 25010:2011: System and Software Quality Models,” ISO/IEC, 2011.
[3]	R. S. Pressman, Software Engineering: A Practitioner’s Approach, 8th ed. New York: McGraw-Hill, 2014.
[4]	I. Sommerville, Software Engineering, 10th ed. Harlow: Pearson Education Limited, 2016.
[5]	VMware Inc., “Spring Boot Reference Documentation,” [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/. [Accessed: Apr. 2026].
[6]	Oracle Corporation, “MySQL 8.0 Reference Manual,” [Online]. Available: https://dev.mysql.com/doc/. [Accessed: Apr. 2026].
[7]	Google LLC, “Google Maps Platform Documentation,” [Online]. Available: https://developers.google.com/maps/documentation. [Accessed: Apr. 2026].
[8]	Stripe Inc., “Stripe API Reference,” [Online]. Available: https://stripe.com/docs/api. [Accessed: Apr. 2026].
[9]	PayPal Holdings Inc., “PayPal REST API,” [Online]. Available: https://developer.paypal.com/docs/api/. [Accessed: Apr. 2026].
[10]	W3C, “Web Content Accessibility Guidelines (WCAG) 2.1,” [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: Apr. 2026].

Appendix A: Client Agreement

A.1 Client Requirement Sign-off

The following sign-off confirms that the client representative has reviewed and approved the scope, functional requirements, and features documented in this SRS. No development activities shall commence prior to receipt of this signed approval.

Note: Any changes to the approved requirements after this sign-off must be submitted as a formal Change Request and re-approved by both parties before implementation.

A.2 Sample Agreement

Date: 20 March 2026

To Whom It May Concern,

This letter confirms that we, Tharindu Rajapakshe, representing Scooter Rent Kandy, have agreed to collaborate with Jojeevan (SE/2021/013), Kajanan (SE/2021/032), Gobinath (SE/2021/045), and Mohamad (SE/2021/048) of the Faculty of Science, University of Kelaniya, Sri Lanka, for the purpose of developing a software-based system as part of their academic System Design Project.

1. Project Title

Scooter Rental Management System

2. Background and Problem Statement

Currently, the business manages scooter rental operations manually using paper-based agreements and basic record-keeping methods. There is no centralized system to manage bookings, customer details, or vehicle tracking. This results in several operational issues:

- Difficulty in managing and retrieving rental records.
- Lack of proper tracking of rental duration and mileage.
- Risk of data loss or errors in manual records.

- Inefficiency in handling customer requests and booking enquiries.

A digital system is therefore required to improve efficiency, accuracy, and the overall customer experience.

3. Purpose of the Proposed System

- To provide an online booking system for scooter rentals.
- To maintain digital records of customers and rental activities.
- To track rental duration, distance, and vehicle usage via GPS.
- To enable automated notifications through SMS and email.
- To improve overall management and operational efficiency.

4. Scope of Collaboration

The client agrees to:

- Provide necessary information related to current business processes.
- Participate in requirements discussions and clarification meetings.
- Review and validate the Software Requirement Specification (SRS).
- Provide timely feedback on system design and prototype demonstrations.

The student developers agree to:

- Maintain the confidentiality of all sensitive business information.
- Use all provided data solely for academic purposes.
- Provide regular updates on project progress.
- Communicate any proposed changes to the project scope in a timely manner.

5. Nature of the Project

This project is undertaken solely for academic purposes. The University of Kelaniya and the student developers are not liable for any direct or indirect damages, data loss, or financial impact arising from the use of this software.

6. Confidentiality

All confidential information shared during this project will be used strictly for academic purposes and will not be disclosed to third parties without prior consent. This agreement remains valid for two (2) years after project completion.

7. Intellectual Property

All software, code, and documentation developed during this project will remain the property of the student developers and/or the University of Kelaniya. The client is granted permission to use the system for internal evaluation purposes only; commercial use requires prior written approval.

8. Term and Termination

This agreement will remain valid from the date of signing until the final submission of the project. Either party may terminate the agreement with written notice of fourteen (14) days.

We hereby confirm that the above project request is genuine and that the proposed solution is required.

A.3 Copy of Original Agreement

The following is a complete typed copy of the original client agreement signed by Mr. Tharindu Rajapakshe on behalf of Scooter Rent Kandy on 20 March 2026. The signed original is retained by the development team and is available for inspection upon request.

Date: 20 March 2026

To Whom It May Concern,

This letter confirms that we, Tharindu Rajapakshe, representing Scooter Rent Kandy, have agreed to collaborate with the following student developers of the Faculty of Science, University of Kelaniya, for the purpose of developing a software-based system as part of their academic System Design Project (SENG 31242):

Student Name	Student Number	Role in Project
Jojeevan	SE/2021/013	Team Lead / Backend Developer
Kajanan Sivalingam	SE/2021/032	Frontend Developer
Gobinath Ehamparam	SE/2021/045	Database Engineer
Mohamed Riyal Mohamed	SE/2021/048	Systems Analyst / Tester

Table 31: Original Agreement – Student Team

1. Project Title

Scooter Rental Management System – Scooter Rent Kandy

2. Background and Problem Statement

The business currently manages scooter rental operations manually using paper-based agreements and handwritten records, resulting in:

- Difficulty in managing and retrieving rental records efficiently.
- Lack of systematic tracking of rental duration and GPS-based mileage.
- Risk of data loss or transcription errors in manual records.
- Inability to provide customers with real-time vehicle availability information.
- Inefficiency in processing payments and issuing rental receipts.

A digital system is therefore required to improve efficiency, accuracy, and the overall customer experience.

3. Purpose of the Proposed System

The proposed system is expected to:

- Provide an online booking platform for scooter and camping gear rentals.
- Maintain complete digital records of customers, bookings, and rental activities.
- Track rental duration and GPS-based distance for fully automated billing.
- Enable automated SMS and email notifications for booking confirmations and alerts.
- Provide an administrative dashboard for fleet, revenue, and report management.

4. Scope of Collaboration

The client agrees to: provide business process information; participate in requirements discussions; review and validate the SRS; provide timely feedback on design demonstrations.

The student developers agree to: maintain confidentiality of sensitive business information; use all provided data solely for academic purposes; provide regular progress updates; communicate all proposed scope changes promptly.

5. Nature of the Project

This project is undertaken solely for academic purposes. The University of Kelaniya and the student developers disclaim all commercial warranties. The system is an academic proof-of-concept and shall not be deployed commercially without independent professional review.

6. Confidentiality

All confidential information shared during this project shall be used strictly for academic purposes. These obligations survive for two (2) years following project completion.

7. Intellectual Property

All software, code, documentation, and design artefacts remain the intellectual property of the student developers and the University of Kelaniya. The client is granted a non-exclusive, royalty-free licence for internal evaluation and testing purposes only.

8. Term and Termination

This agreement shall remain valid from the date of signing until final submission of all project deliverables. Either party may terminate by providing 14 days' written notice.

We hereby confirm that the above project request is genuine and that the proposed solution is required by our business.

Client Name: Tharindu Rajapakshe

Designation: Business Owner

Organisation: Scooter Rent Kandy, Kandy, Sri Lanka

Contact Information: 074 077 8227 / 076 372 7753

Date of Signing: 20 March 2026

Signature: _____



Official Company Seal:

A.4 Official Client Feedback Summary – SRS and SDS Version 1.0

This section presents the official record of client feedback received from Mr. Tharindu Rajapakshe upon his review of Version 1.0 of the SRS and SDS, received on 30 April 2026. The feedback has been documented in formal academic language in accordance with the project communication protocol established at the commencement of SENG 31242.

Field	Details
Date of Feedback	April 30, 2026
Stakeholder	Tharindu Rajapakshe – Client, Scooter Rent Kandy
Documents Reviewed	SRS Version 1.0 and SDS Version 1.0
Communication Channel	In-person meeting with verbal and written confirmation
Recorded By	Development Team (SE/2021/013, 032, 045, 048)

Table 32: Official Client Feedback Summary – Metadata

1. Summary of Feedback

The client expressed overall satisfaction with the quality and direction of the work, providing a green light for the current development phase to proceed without mandatory revisions to the core requirements or design artefacts.

2. Key Stakeholder Observations

2.1 Approval of Progress

The client explicitly stated: "The work is good. Let's continue it." This constitutes a formal verbal endorsement of all current project artefacts and authorises the development team to proceed to the implementation phase.

2.2 Technical Complexity of Documentation

The client observed that the technical depth of the SRS and SDS creates a barrier to forming an intuitive, non-technical understanding of the system's end-to-end workflow. This is

recorded as constructive feedback and does not constitute a rejection of any design decision or requirement.

2.3 Open Collaboration and Resource Availability

The client confirmed continued supportive engagement and offered to provide any additional information or clarification required to maintain project momentum.

3. Action Plan in Response to Feedback

#	Action	Description	Target
1	Executive Summaries	Add a one-page Plain English summary at the start of Chapters 2 and 3, explaining the system without code or database terminology.	Ch 2 & 3
2	Enhanced Visualization	Add high-level user journey flowcharts illustrating the path from browsing through booking, payment, and GPS-tracked rental.	Ch 2 & 3
3	Glossary of Terms	Add a technical glossary in the Appendix defining: NFR, API, GPS, JWT, RBAC, BCNF, ORM.	Appendix

4. Client Acknowledgement

Field	Details
Client Name	Tharindu Rajapakshe
Organisation	Scooter Rent Kandy, Kandy, Sri Lanka
Feedback Date	30 April 2026
SRS v1.0 Status	Approved – no mandatory revisions required
SDS v1.0 Status	Approved – no mandatory revisions required
Action Plan Accepted	Yes – client verbally approved all three improvements
Client Signature	_____
Date	_____

Table 33: Official Client Feedback Summary – Acknowledgement

Chapter 3

Software Design Specification

3.1 Architectural Design

This chapter presents the high-level system architecture, justification for the chosen design pattern, and the scalability and maintainability strategies employed in the Scooter Rent Kandy system. All design decisions are directly traceable to the functional and nonfunctional requirements documented in the approved SRS.

3. 1.1 High-Level Architecture

A layered Model-View-Controller (MVC) architecture has been adopted for the Scooter Rent Kandy system. The architecture is decomposed into six distinct layers, each with welldefined responsibilities. This separation of concerns ensures that changes to one layer do not propagate uncontrollably to others, thereby improving maintainability and testability.

Figure 1: Scooter Rent Kandy – High-Level System Architecture



Figure 1: Scooter Rent Kandy – High-Level System Architecture Diagram

Layer	Technology / Component	Responsibility
Presentation	React.js, Google Maps Widget, Stripe.js	Renders the user interface; handles user interactions; communicates with the API over HTTPS.
API Gateway	AWS ALB, Spring Security, TLS 1.3	Terminates SSL; authenticates JWT tokens; enforces RBAC; routes requests to business logic.
Business Logic	Spring Boot Services (BookingService, GPSBillingService, PaymentService, ReportService)	Implements all business rules; orchestrates calls to the data layer and external services.
Data Access	Spring Data JPA, Hibernate ORM, HikariCP	Abstracts database queries using the Repository pattern; manages connection pooling.
Database	Amazon RDS PostgreSQL 16 (MultiAZ)	Persists all system data; provides ACID transactions and automated backups.
External Services	Google Maps API, Stripe, AWS SES/SNS, AWS S3	GPS distance calculation, payment processing, email/SMS notifications, and file storage.

3.1.2 Justification for Architecture Choice

The layered MVC pattern has been selected over alternatives such as microservices and monolithic architectures for the following reasons. First, the project scope and team size (four developers) are better suited to a layered monolith, which reduces deployment and operational complexity. Second, MVC clearly separates user interface concerns from business logic and data access, enabling parallel development streams. Third, the architecture can be migrated towards microservices incrementally if the system scales beyond its initial scope. A cloud-native deployment on AWS has been adopted to provide elastic scalability, managed database services, and integrated monitoring without the overhead of on-premises infrastructure management.

The layered MVC pattern was selected over alternative architectures based on the criteria summarized in Table 13 below.

Table 13: Justification for Architecture Choice

Criterion	Rationale
Separation of Concerns	Each layer has a single responsibility, reducing coupling and simplifying maintenance and testing.
Technology Fit	Spring Boot and React.js are mature, well-documented frameworks with extensive community support and ecosystem integration.
Scalability	Individual layers scale independently; the API tier scales via AWS Auto Scaling without affecting other tiers.
Security Enforcement	Security controls (JWT, RBAC) are enforced at the API Gateway layer before requests reach business logic.
Team Familiarity	The development team has prior expertise in Spring Boot and React, reducing onboarding overhead.
Testability	Distinct layers allow unit, integration, and end-to-end tests to target specific components independently.

3.1.3 Scalability and Maintainability Considerations

- Horizontal scaling is achieved through an AWS Auto Scaling Group attached to the Application Load Balancer; additional EC2 instances launch automatically when CPU utilization exceeds 70%.
- The stateless REST API design ensures that any backend instance can serve any request, enabling true horizontal scalability without sticky sessions.

- Database connection pooling via HikariCP is configured to a maximum of 20 connections per instance, preventing connection exhaustion under load.
- Modular Spring Boot service classes follow the Single Responsibility Principle, allowing individual services (e.g., GPSBillingService) to be refactored or extracted without affecting others.
- RESTful API versioning (/api/v1/) enables future breaking changes to be introduced without disrupting existing clients.
- Docker containerization ensures environment parity between development, staging, and production.

3.2.Database Design

This chapter presents the Entity-Relationship (ER) diagram in Chen notation, entity descriptions with full attribute tables, key relationships, and normalization analysis for the Scooter Rent Kandy database schema. The schema is implemented in PostgreSQL 16 and accessed via Spring Data JPA (Hibernate). All entity primary keys use UUID type to ensure globally unique, non-sequential identifiers.

3.2.1 Entity-Relationship (ER) Diagram

The ER diagram below uses Chen notation: rectangles for strong entities, a double rectangle for the weak entity (Booking Gear), ellipses for attributes, underlined ellipses for primary keys, and diamonds for relationships. Cardinality labels (1 and N) are placed on the connecting lines.

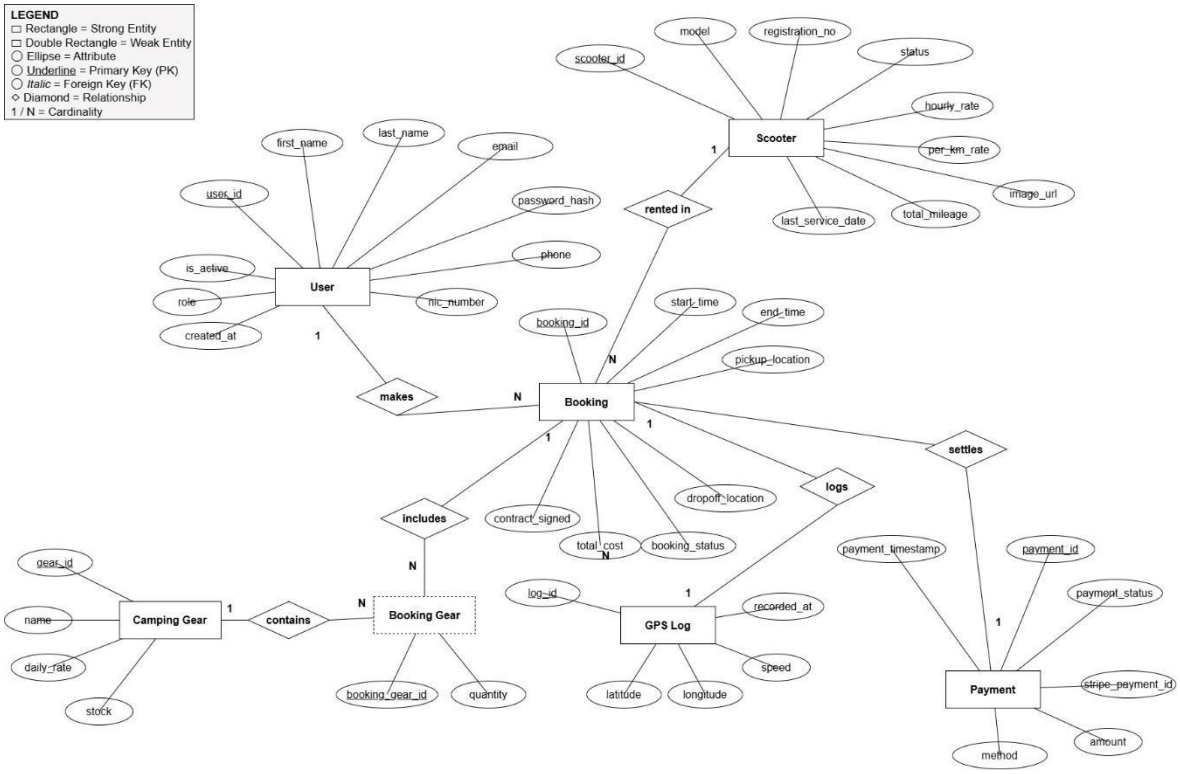


Figure 2: Scooter Rent Kandy – Entity-Relationship (ER) Diagram (Chen Notation)

3.2.2 Entity Descriptions and Attributes

User

The User entity stores information about all registered users of the system, including both customers and administrators.

Attribute	Data Type	Constraint	Description
userId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
firstName	VARCHAR(100)	NOT NULL	User's given name
lastName	VARCHAR(100)	NOT NULL	User's family name
email	VARCHAR(255)	NOT NULL, UNIQUE	Login credential; must be in valid email format
passwordHash	VARCHAR(255)	NOT NULL	BCrypt hashed password (cost factor 12); plain text never stored

phone	VARCHAR(20)	NOT NULL	Mobile contact in international format (+94XXXXXXXXX)
nicNumber	VARCHAR(20)	NOT NULL, UNIQUE	Sri Lanka NIC number; encrypted at rest using AES256
role	ENUM	NOT NULL, DEFAULT=USER	USER | ADMIN | SUPER_ADMIN – determines RBAC access level
isActive	BOOLEAN	DEFAULT TRUE	TRUE = active account; FALSE = blocked by administrator
createdAt	TIMESTAMP	NOT NULL	Account creation timestamp in ISO 8601 UTC format

Scooter

The Scooter entity stores details of every scooter in the rental fleet managed by the administrator.

Attribute	Data Type	Constraint	Description
scooterId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
model	VARCHAR(150)	NOT NULL	Scooter model name (e.g., Honda Dio, Yamaha Ray-ZR)
Attribute	**Data Type**	**Constraint**	**Description**
registrationNo	VARCHAR(30)	NOT NULL, UNIQUE	Official Sri Lanka vehicle registration plate number
status	ENUM	NOT NULL	AVAILABLE | RENTED | MAINTENANCE – real-time fleet status
hourlyRate	DECIMAL(10,2)	NOT NULL, > 0	Rental charge per hour in Sri Lankan Rupees (LKR)

perKmRate	DECIMAL(10,2)	NOT NULL, >= 0	Distance-based charge per kilometer in LKR
imageUrl	VARCHAR(500)	NULLABLE	AWS S3 URL for the scooter image (JPEG/PNG, max 5 MB)
totalMileage	DECIMAL(10,2)	DEFAULT 0	Cumulative distance travelled by the scooter in kilometers
lastServiceDate	DATE	NULLABLE	Date of most recent maintenance service (ISO 8601)

Booking

The Booking entity records every rental transaction initiated by a user, from reservation through to completion or cancellation.

Attribute	Data Type	Constraint	Description
bookingId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
userId (FK)	UUID	NOT NULL	References User.userId
scooterId (FK)	UUID	NOT NULL	References Scooter.scooterId
startTime	TIMESTAMP	NOT NULL	Rental start date and time (ISO 8601 UTC)
endTime	TIMESTAMP	NULLABLE	Rental end date/time; NULL if rental is still active
pickupLocation	VARCHAR(300)	NOT NULL	GPS address string of the scooter pickup point
dropLocation	VARCHAR(300)	NULLABLE	GPS address of return point; NULL until rental ends
Attribute	**Data Type**	**Constraint**	**Description**
status	ENUM	NOT NULL	PENDING | ACTIVE | COMPLETED | CANCELLED
totalCost	DECIMAL(10,2)	NULLABLE	Calculated final cost in LKR; NULL until booking completes

contractSigned	BOOLEAN	DEFAULT FALSE	TRUE = digital rental contract accepted by the user

Payment

The Payment entity records the financial transaction associated with each confirmed booking.

Attribute	Data Type	Constraint	Description
paymentId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
bookingId (FK)	UUID	NOT NULL, UNIQUE	References Booking.bookingId (one payment per booking)
stripePaymentId	VARCHAR(255)	NULLABLE	Stripe PaymentIntent reference ID
amount	DECIMAL(10,2)	NOT NULL, > 0	Total amount charged to the user in LKR
method	ENUM	NOT NULL	CARD | ONLINE – payment method used at checkout
status	ENUM	NOT NULL	PENDING | SUCCESS | FAILED | REFUNDED
paidAt	TIMESTAMP	NULLABLE	Date/time payment was successfully processed (ISO 8601 UTC)

GPS Log

The GPS Log entity records real-time location data transmitted by the scooter GPS device during an active rental, used for distance calculation and speed violation monitoring.

Attribute	Data Type	Constraint	Description
logId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
bookingId (FK)	UUID	NOT NULL	References Booking.bookingId for the active rental
latitude	DECIMAL(9,6)	NOT NULL	GPS latitude coordinate in WGS84 decimal degrees

longitude	DECIMAL(9,6)	NOT NULL	GPS longitude coordinate in WGS84 decimal degrees
speed	DECIMAL(5,2)	NOT NULL, >= 0	Speed of the scooter in km/h at the time of logging
recordedAt	TIMESTAMP	NOT NULL	ISO 8601 UTC; logged every 30 seconds by the GPS device

Camping Gear

The Camping Gear entity stores details of optional camping equipment that users may add to a scooter booking.

Attribute	Data Type	Constraint	Description
gearId (PK)	UUID	NOT NULL, UNIQUE	System-generated surrogate primary key
name	VARCHAR(200)	NOT NULL	Gear name (e.g., Camping Tent, Gas Cooker, LED Light)
dailyRate	DECIMAL(10,2)	NOT NULL, > 0	Daily rental charge per item in LKR
stock	INTEGER	NOT NULL, >= 0	Number of units currently available in inventory

3.2.3 Key Relationships

User → Booking (1 to N): A user may place zero or many bookings. Each booking belongs to exactly one user.

Scooter → Booking (1 to N): A scooter may appear in zero or many bookings sequentially. Only one booking may be ACTIVE per scooter at any time (enforced by a partial unique index on scooterId WHERE status = ACTIVE).

Booking → Payment (1 to 1): Each confirmed booking has exactly one associated payment record (enforced by UNIQUE constraint on Payment.bookingId).

Booking → GPS Log (1 to N): An active booking generates many GPS log entries, recorded every 30 seconds by the scooter device.

Booking ↔ Camping Gear (M to N): A booking may include zero or many camping gear items; represented by a Booking Gear junction (weak) entity with a quantity attribute.

3.2.4 Normalization

The database schema has been designed to comply with Third Normal Form (3NF). The table below summaries how each normal form has been addressed.

Normal Form	Requirement	Application to Scooter Rent Kandy
1NF	Atomic values; no repeating groups	All attributes store single atomic values. Camping gear items stored in a separate CampingGear entity referenced via the BookingGear junction table.
2NF	Full functional dependency on PK	All non-key attributes are fully functionally dependent on their single-column UUID primary key. No partial dependencies exist.
3NF	No transitive dependencies	Billing rates stored on Scooter, not duplicated in Booking. Payment method is an ENUM on Payment entity, eliminating transitive dependency.
BCNF	Every determinant is a candidate key	Scooter availability computed from Booking records; no derived attributes stored that could violate BCNF.

A controlled denormalization is applied to the total Cost column of the Booking entity. Although this value can be computed from GPS log data and rates, it is persisted after calculation for immutable billing record and audit purposes. This is a documented exception to 3NF.

3.3: Class Diagram

The UML Class Diagram below models the core domain classes of the Scooter Rent Kandy system. Each class corresponds directly to a JPA entity or service class in the Spring Boot application. The diagram presents all attributes with types and visibility modifiers (- private, + public), all method signatures with parameters and return types, and all relationships with multiplicity labels. The colour scheme matches the project draw.io standard.

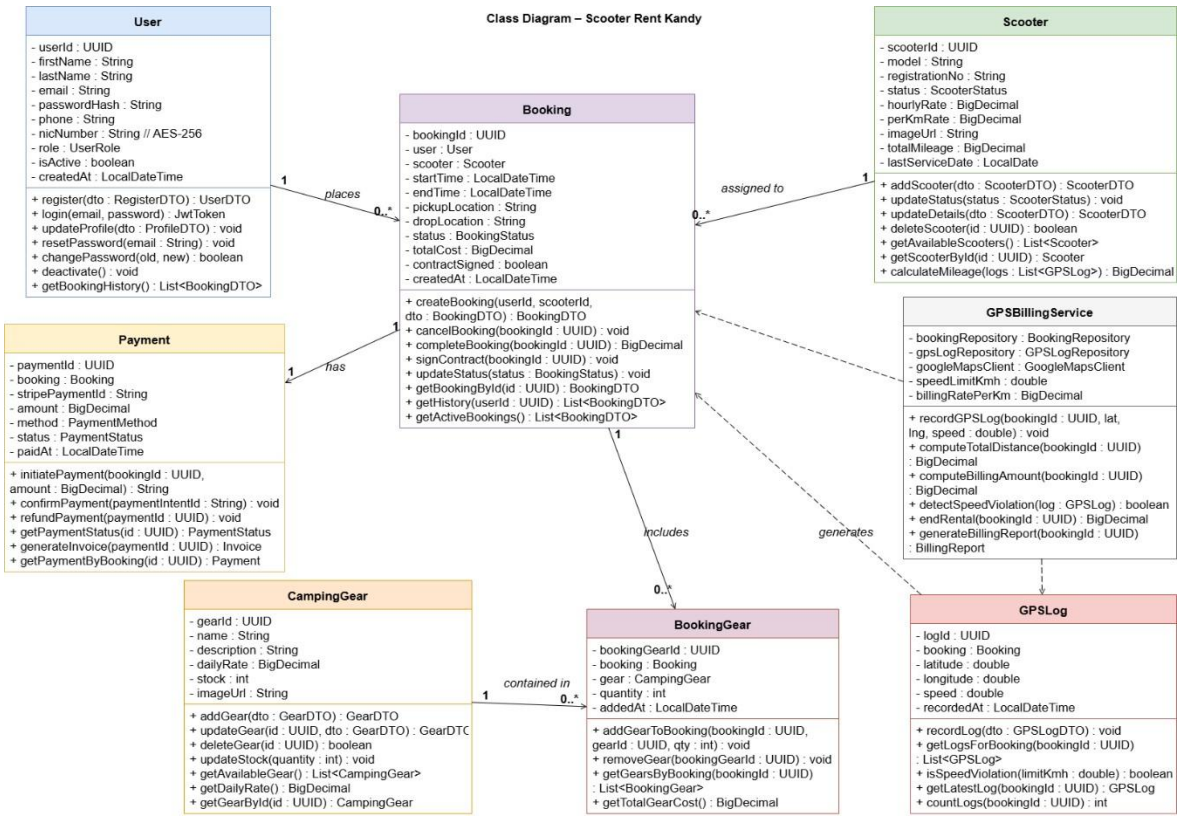


Figure 3: Scooter Rent Kandy – UML Class Diagram

3.1 Domain Class Descriptions

Class	Responsibility	Key Relationships
User	Represents a registered customer or administrator. Manages authentication, profile, and booking history.	1..N Booking (places)

Scooter	Represents a rental vehicle. Tracks availability, pricing rates, and cumulative mileage.	1..N Booking (assigned to)
Booking	Central entity capturing the full rental lifecycle from PENDING	References User and Scooter; has 1 Payment, N GPSLog, N BookingGear
Class	**Responsibility**	**Key Relationships**
	through ACTIVE to COMPLETED or CANCELLED.	
Payment	Records the financial transaction for a booking. References Stripe PaymentIntent; card data never stored locally.	1..1 Booking
CampingGear	Represents optional rental equipment available as booking add-ons.	M..N Booking via BookingGear junction
BookingGear	Junction entity implementing M:N between Booking and CampingGear, with a quantity attribute.	N..1 Booking, N..1 CampingGear
GPSBillingService	Processes GPS logs, computes distance via Google Maps Distance Matrix API, calculates final rental bill.	<<uses>> Booking and GPSLog
GPSLog	Stores real-time positional data emitted by the GPS device every 30 seconds during an active rental.	N..1 Booking

3.2 Enumerations

Enumeration	Values	Usage
UserRole	USER, ADMIN, SUPER_ADMIN	Assigned to User to control RBAC permissions at method level.
ScooterStatus	AVAILABLE, RENTED, MAINTENANCE	Controls scooter visibility in browse results and booking eligibility.

BookingStatus	PENDING, ACTIVE, COMPLETED, CANCELLED	Tracks the lifecycle state of every booking record.
PaymentMethod	CARD, ONLINE	Records the payment method used at checkout.
PaymentStatus	PENDING, SUCCESS, FAILED, REFUNDED	Tracks the state of a payment from initiation to settlement.

3.3 Relationships and Multiplicity

User (1) ——— (0..*) Booking : A user may have zero or many bookings over their account lifetime.

Scooter (1) ——— (0..*) Booking : A scooter appears in zero or many bookings sequentially over its fleet lifetime.

Booking (1) ——— (1) Payment : Every completed booking has exactly one payment record.

Booking (1) ——— (0..*) GPS Log : An active booking generates many GPS log entries every 30 seconds.

Booking (0..*) ─M:N─ (0..*) CampingGear : Represented by the BookingGear junction class with a quantity attribute.

GPSBillingService ──<<uses>>── Booking and GPSLog : Dependency (dashed arrow); the service reads but does not own these entities.

3.4: Sequence Diagrams

Sequence diagrams are provided for all four major use cases. Each diagram follows UML 2.5 notation: solid arrows represent synchronous calls, dashed arrows represent return messages, and activation bars indicate periods of component execution.

3.4.1 User Registration

This diagram illustrates the complete registration flow from form submission through clientside validation, server-side DTO validation, BCrypt password hashing,

AES-256 NIC encryption, database insertion, and welcome email dispatch via AWS SES.

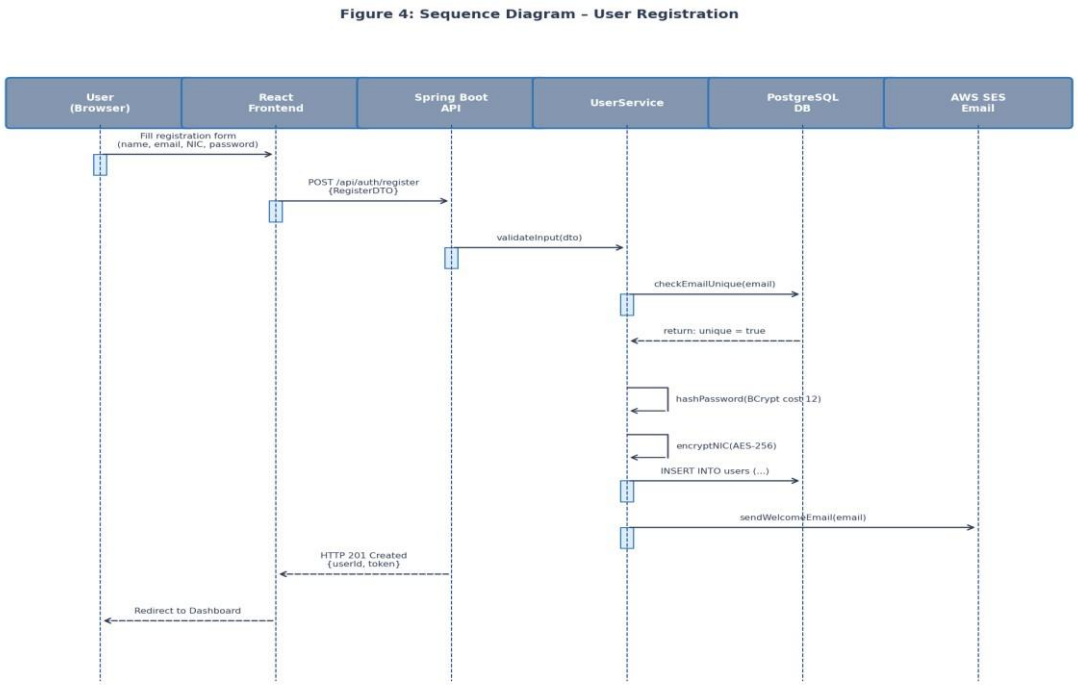


Figure 4: Sequence Diagram – User Registration

#	From	To	Message / Action
1	User (Browser)	React Frontend	Fill registration form and submit
2	React Frontend	Spring Boot API /auth/register	POST {name, email, phone, NIC, password}
3	Spring Boot API	UserService	validateInput(dto)
4	UserService	PostgreSQL DB	checkEmailUnique(email)
#	From	To	Message / Action
5	PostgreSQL DB	UserService	return: unique = true
6	UserService	–	hashPassword(BCrypt cost 12) [self-call]
7	UserService	–	encryptNIC(AES-256) [self-call]
8	UserService	PostgreSQL DB	INSERT INTO users (...)

9	UserService	AWS SES	sendWelcomeEmail(email)
10	Spring Boot API	React Frontend	HTTP 201 Created {userId, token}
11	React Frontend	User (Browser)	Redirect to Dashboard

3.4.2 Scooter Booking

This diagram covers the complete booking flow: scooter availability validation, booking record creation, Stripe PaymentIntent creation, webhook confirmation upon payment success, booking activation, and SMS confirmation via AWS SNS.

Figure 5: Sequence Diagram – Scooter Booking

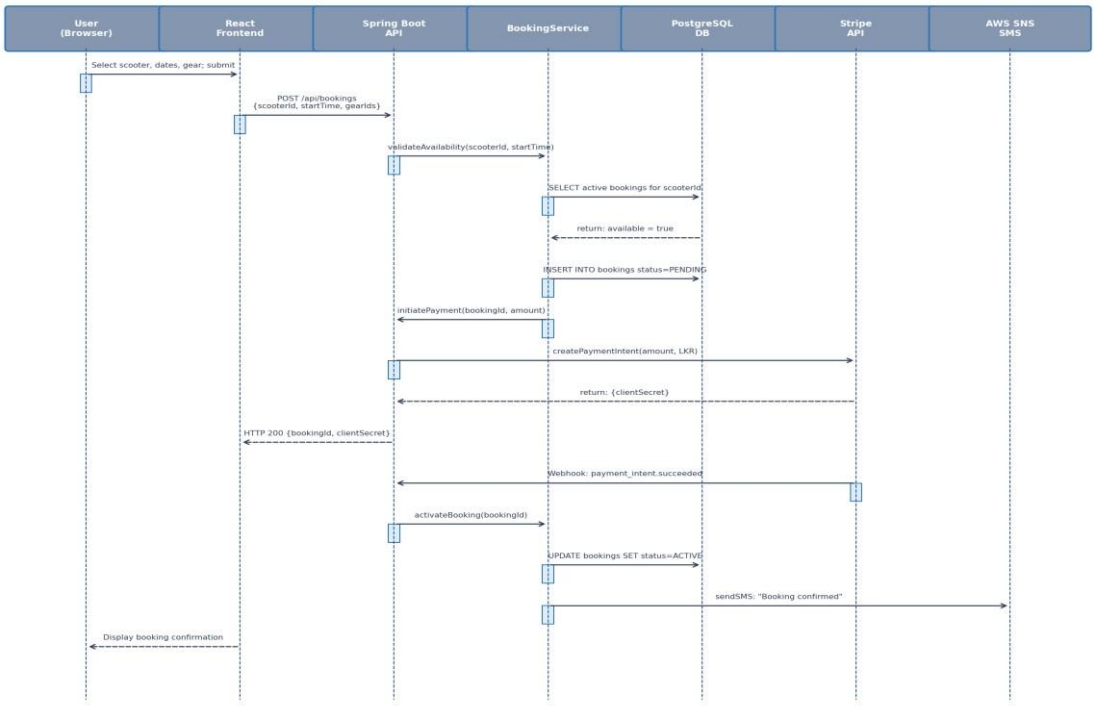


Figure 5: Sequence Diagram – Scooter Booking

#	From	To	Message / Action
1	User (Browser)	React Frontend	Select scooter, dates, gear; submit booking
2	React Frontend	Spring Boot API /bookings	POST {scooterId, startTime, gearIds}
3	Spring Boot API	BookingService	validateAvailability(scooterId, startTime)

4	BookingService	PostgreSQL DB	SELECT active bookings for scooterId
5	PostgreSQL DB	BookingService	return: available = true
6	BookingService	PostgreSQL DB	INSERT INTO bookings (...) status=PENDING
7	BookingService	PaymentService	initiatePayment(bookingId, amount)
8	PaymentService	Stripe API	createPaymentIntent(amount, currency=LKR)
9	Stripe API	PaymentService	return: clientSecret
10	Spring Boot API	React Frontend	HTTP 200 {bookingId, clientSecret}
11	React Frontend	Stripe.js	confirmCardPayment(clientSecret)
12	Stripe	Spring Boot Webhook /webhooks/stripe	payment_intent.succeeded event
13	Spring Boot Webhook	BookingService	activateBooking(bookingId)
14	BookingService	PostgreSQL DB	UPDATE bookings SET status=ACTIVE
15	BookingService	AWS SNS	sendSMS(userId, "Booking confirmed")
16	React Frontend	User (Browser)	Display booking confirmation

3.4.3 GPS-Based Billing Calculation

This diagram illustrates real-time GPS data collection (POST every 30 seconds), speed violation detection, end-rental trigger, total distance computation via Google Maps Distance Matrix API, billing calculation, and invoice generation.

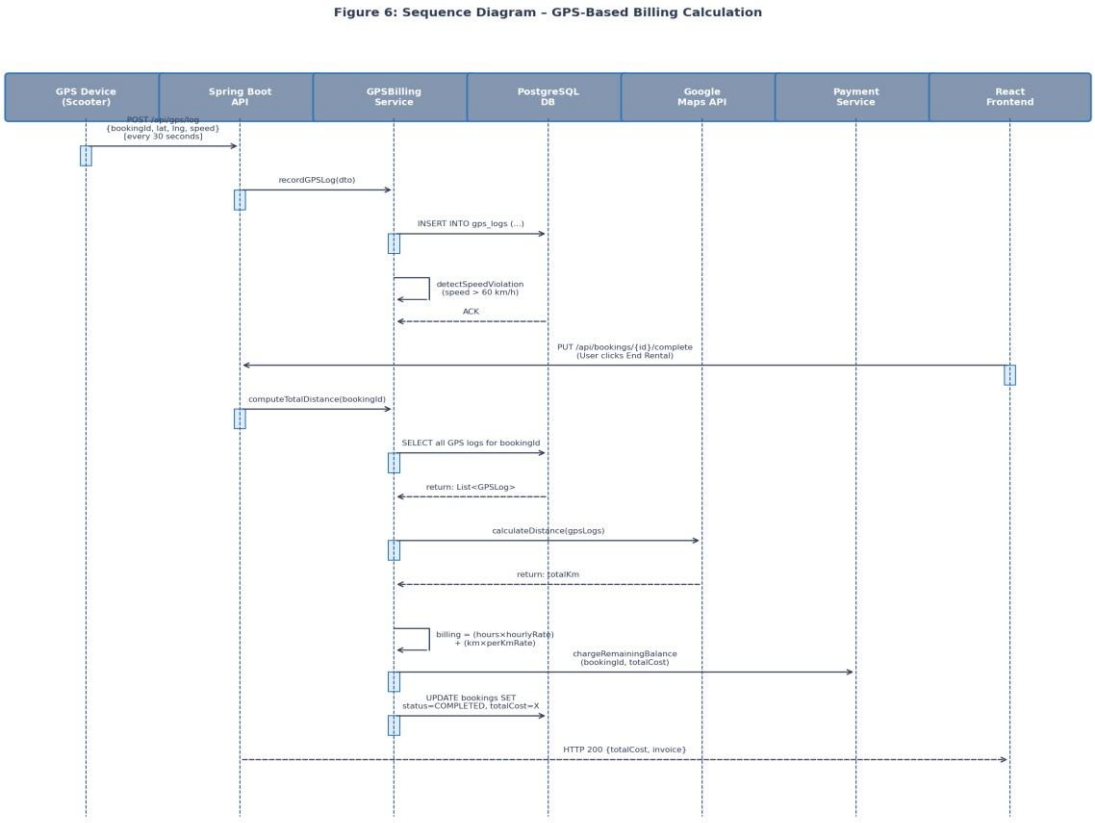


Figure 6: Sequence Diagram – GPS-Based Billing Calculation

#	From	To	Message / Action
1	Scooter GPS Device	Spring Boot API /gps/log	POST {bookingId, lat, lng, speed, timestamp} [every 30s]
2	Spring Boot API	GPSBillingService	recordGPSLog(dto)
3	GPSBillingService	PostgreSQL DB	INSERT INTO gps_logs (...)
4	GPSBillingService	–	detectSpeedViolation(speed, 60 km/h) [self-call]
5	User	React Frontend	Click "End Rental"
6	React Frontend	Spring Boot API /bookings/{id}/complete	PUT request
7	Spring Boot API	GPSBillingService	computeTotalDistance(bookingId)
8	GPSBillingService	Google Maps API	calculateDistance(gpsLogs)

9	Google Maps API	GPSBillingService	return: totalKm
10	GPSBillingService	–	billing = (hours × hourlyRate) + (km × perKmRate)
#	**From**	**To**	**Message / Action**
11	GPSBillingService	PaymentService	chargeRemainingBalance(bookingId, totalCost)
12	BookingService	PostgreSQL DB	UPDATE bookings SET status=COMPLETED, totalCost=X
13	Spring Boot API	React Frontend	HTTP 200 {totalCost, invoice}
14	React Frontend	User (Browser)	Display invoice and receipt

3.4.4 Admin Report Generation

This diagram covers admin report generation: JWT role validation, data aggregation from PostgreSQL, report DTO construction, optional PDF/CSV export with upload to AWS S3, and download URL delivery.

Figure 7: Sequence Diagram – Admin Report Generation

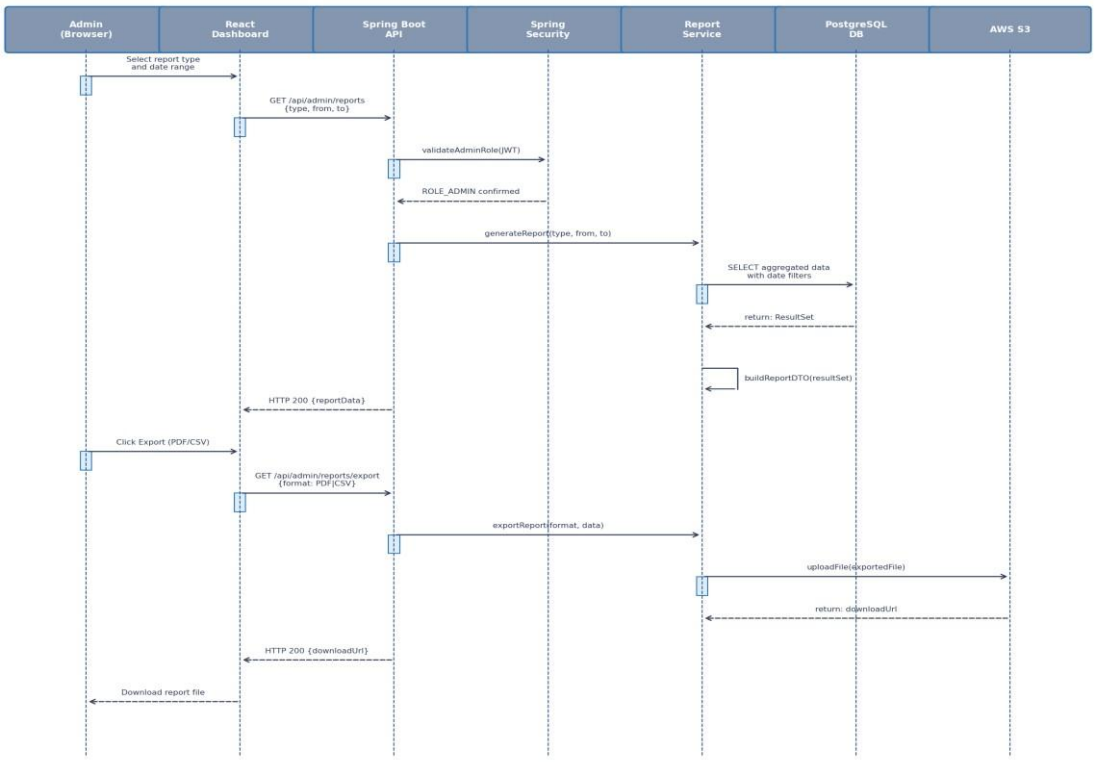


Figure 7: Sequence Diagram – Admin Report Generation

#	From	To	Message / Action
1	Admin (Browser)	React Admin Dashboard	Select report type and date range
2	React Admin Dashboard	Spring Boot API /admin/reports	GET {type, from, to}
3	Spring Boot API	Spring Security	validateAdminRole(JWT)
4	Spring Security	Spring Boot API	return: ROLE_ADMIN confirmed
5	Spring Boot API	ReportService	generateReport(type, from, to)
6	ReportService	PostgreSQL DB	SELECT aggregated data with date filters
7	PostgreSQL DB	ReportService	return: ResultSet
8	ReportService	–	buildReportDTO(resultSet) [selfcall]
9	Spring Boot API	React Admin Dashboard	HTTP 200 {reportData: [...]}
10	Admin (Browser)	Admin	View report or click Export
11	React Admin Dashboard	Spring Boot API /admin/reports/export	GET {format: PDF|CSV}
12	Spring Boot API	ReportService	exportReport(format, data)
13	ReportService	AWS S3	uploadFile(exportedFile)
14	Spring Boot API	React Admin Dashboard	HTTP 200 {downloadUrl}
15	React Admin Dashboard	Admin (Browser)	Download report file

3.5: UI/UX Design

This chapter describes the user interface design principles, key screen descriptions, accessibility considerations, and the navigation flow for the Scooter Rent Kandy system. All screens are implemented using React.js with a responsive CSS Grid and Flexbox layout adapting to desktop (1280px+), tablet (768px), and mobile (375px) viewports.

3.5.1 Design Principles

- Consistency: Common components (navigation bars, buttons, form fields) are reused across all screens with identical styling derived from the brand identity (gold #B8860B and dark background with white accents).
- Feedback: Every user action receives immediate visual feedback including loading spinners, success toast notifications, and error banners to indicate system state clearly.
- Simplicity: Each screen focuses on a single primary task; secondary actions are placed in overflow menus or secondary panels to reduce cognitive load.
- Accessibility: All screens target WCAG 2.1 Level AA compliance, as detailed in the table below.
- Responsiveness: The interface adapts to desktop (1280px+), tablet (768px), and mobile (375px) viewports via CSS media queries.

3.5.2 Screen Descriptions and Accessibility Considerations

The following table summarises the key screens, target roles, primary UI elements, and WCAG 2.1 Level AA accessibility considerations for each screen.

Table 22: Screen Descriptions and Accessibility Considerations

Screen / Module	Role	Key UI Elements	Accessibility Considerations
Registration / Login	Public	Email, password, OTP, forgot-password link	ARIA labels; aria-live error messages; contrast ≥ 4.5:1
Scooter Browse & Search	User	Grid of scooter cards with image, model, price/hr, badge; filter sidebar	Keyboard-navigable filter; alt text on images; focus indicators
Booking Wizard	User	Date-time picker, gear checklist, Google Maps pickup, contract checkbox	ARIA progress bar; fieldset/legend grouping on each step
Digital Contract	User	Inline PDF viewer, signature canvas, accept/decline buttons	Zoom controls; text-based fallback for motor-impaired users
Payment Page	User	Stripe embedded card element, order summary sidebar	Stripe Elements fully accessible; inline error messages per field
User Dashboard	User	Active bookings list, booking history, notification bell	Tables with th scope; notification count via aria-label
GPS Tracking View	User	Google Maps real-time marker, distance counter, speed indicator	Text-based status panel; speed shown as colour + numeric value
Admin Dashboard	Admin	KPI cards, GPS map, revenue chart, fleet grid	Charts with data table alternative; alerts use aria-live=polite
Reports Page	Admin	Date-range filter, export PDF/CSV, sortable data tables	Keyboard-accessible export buttons; tables with ARIA sort attributes

3.5.3 User Navigation Flow

The navigation flow diagram below illustrates the complete journeys for both User and Admin roles. Role-based routing is enforced at the React Router level using JWT claim verification. Unauthenticated users are redirected to /login; unauthorized access attempts return HTTP 403 Forbidden.

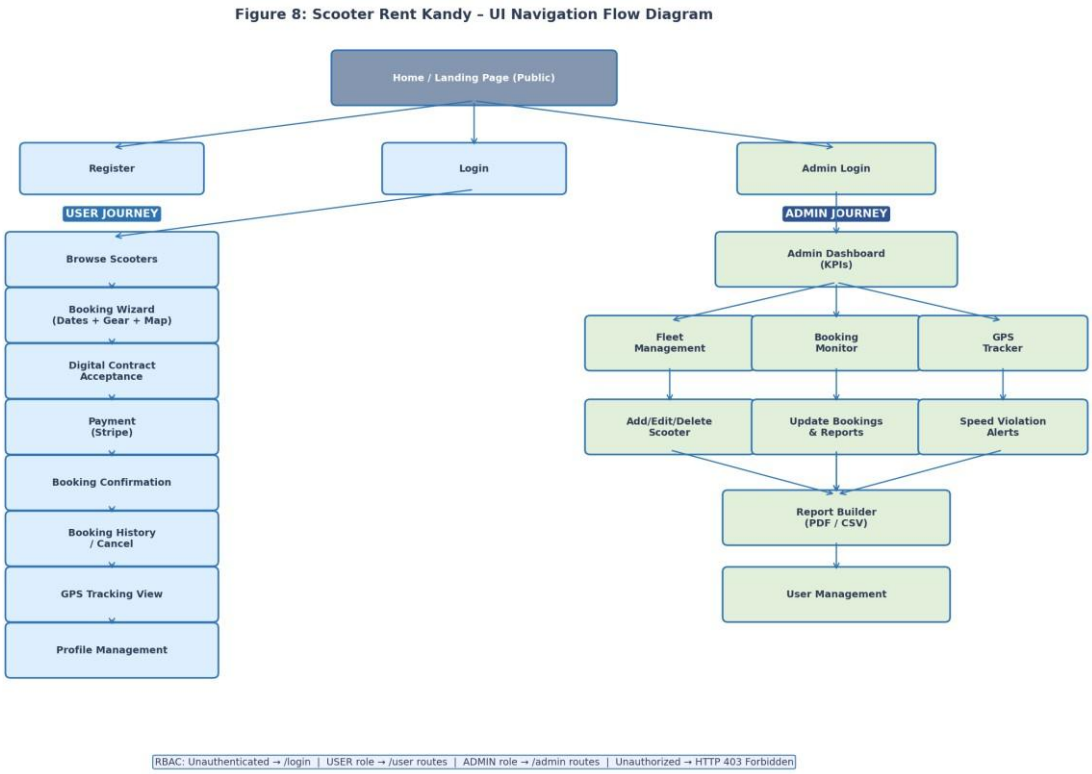


Figure 8: Scooter Rent Kandy – UI Navigation Flow Diagram

An unauthenticated user lands on the Home page and may proceed to Registration or Login. Upon successful login, a USER role is directed to the User Dashboard, from which Scooter Browse, Booking Wizard, Booking History, GPS Tracking View, and Profile screens are accessible. An ADMIN role is directed to the Admin Dashboard, from which Fleet Management, Booking Monitor, GPS Tracker, Report Builder, and User Management are accessible. A logout action from any screen returns the user to the Home page.

3.6: Security Design

The Scooter Rent Kandy system implements a defense-in-depth security strategy with independent protection layers: Network Security (AWS WAF + ALB), Transport Security (TLS 1.3), Application Security (JWT + Spring Security + RBAC), Data Security (AES-256 + BCrypt), and Audit/Monitoring (Spring AOP + AWS CloudWatch). A breach of one layer does not compromise the entire system.

3.6.1 Authentication Mechanism

Authentication is implemented using JSON Web Tokens (JWT) issued by Spring Security upon successful credential verification. The access token has a validity period of one (1) hour. A refresh token, stored in an HttpOnly, SameSite=Strict cookie to prevent JavaScript access, is valid for seven (7) days and is used to obtain a new access token transparently without requiring the user to re-login.

- Multi-factor authentication via OTP delivered by AWS SNS SMS is required for all ADMIN and SUPER_ADMIN logins, providing an additional protection layer for privileged accounts.
- Failed login attempts are rate-limited to 5 per minute per IP address to prevent brute-force attacks.
- The private RSA key used for JWT signing (RS256 algorithm) is stored in AWS

Secrets Manager and is never embedded in application code or environment files.

3.6.2 Security Design Summary

Security Aspect	Mechanism	Details
Authentication	JWT (RS256) + Spring Security	Access tokens expire after 1 hour; refresh tokens stored in HttpOnly cookie (7-day expiry).
Authorization	RBAC – Spring Security @PreAuthorize	Roles: USER, ADMIN, SUPER_ADMIN. Method-level annotations enforce access control.
Password Storage	BCrypt (cost factor 12)	Passwords never stored in plain text; BCrypt hashing applied on registration and password reset.
Data in Transit	TLS 1.3 (HTTPS enforced)	All endpoints served over HTTPS. HTTP → HTTPS permanent redirect (301) at the load balancer.
Data at Rest	AES-256 encryption	Sensitive fields (NIC numbers, Stripe tokens) encrypted at application layer; RDS volume encrypted via AWS KMS.
Input Validation	Server-side + Client-side	Jakarta Bean Validation (JSR-380) on all DTOs; regex whitelisting for GPS coordinates, NIC, phone.
SQL Injection	Parameterized queries via JPA	No raw SQL string concatenation; all queries use prepared statements via Spring Data JPA/Hibernate.
XSS Prevention	Content Security Policy headers	CSP headers restrict inline scripts; React escapes all dynamic content by default.

Payment Security	PCI DSS via Stripe tokenization	Card data never touches our servers; Stripe PaymentIntent API used for all transactions.
Audit Logging	Spring AOP + AWS CloudWatch	All admin actions, login events, and payment events logged with timestamps, IP, and user IDs.

3.6.3 Input Validation and Sanitization Strategy

All input data is validated at two independent levels. At the client side, React form libraries enforce field-level rules (required fields, email format, numeric ranges for GPS coordinates, password strength). At the server side, Jakarta Bean Validation annotations applied to DTOs enforce constraints declaratively. Input exceeding defined length limits is rejected with HTTP 400 Bad Request. All string inputs are stripped of leading and trailing whitespace before processing. File uploads (scooter images) are restricted to JPEG/PNG formats with a maximum size of 5 MB, validated at both the React frontend and the Spring Boot controller layer.

3.6.4 Encryption Standards

Data Category	Standard	Implementation
Data in Transit	TLS 1.3	Enforced via AWS ALB; HTTP permanently redirected to HTTPS (301); HSTS header enabled
Passwords at Rest	BCrypt (cost=12)	Applied in Spring Security PasswordEncoder before any database write
Sensitive PII at Rest	AES-256 GCM	Application-layer encryption via AWS KMSmanaged keys; defense in depth over RDS encryption
Database Volume	AES-256	AWS RDS storage encryption using AWS KMS-managed keys
JWT Signature	RS256 (RSASHA256)	Private key in AWS Secrets Manager; public key distributed to API instances
S3 Objects	AES-256 SSE-S3	Server-side encryption applied automatically to all S3 objects

3.7: Deployment Design

This chapter describes the AWS hosting plan, environment specifications, deployment network architecture, and CI/CD pipeline. All resources are hosted on Amazon Web Services in the Asia Pacific (Singapore) ap-southeast-1 region to minimize latency for the primary user base in Sri Lanka.

3.7.1 Hosting Plan

Component	AWS Service	Specification
Frontend	AWS S3 + CloudFront	React build artefacts hosted on S3; served globally via CloudFront CDN with TLS 1.3 at edge.
Backend API	AWS EC2 (t3.medium)	Spring Boot JAR deployed on EC2 within a private VPC subnet; Auto Scaling Group (min 1, max 4 instances).
Database	Amazon RDS PostgreSQL 16	Multi-AZ deployment for high availability; automated daily snapshots retained for 7 days.
File Storage	Amazon S3	Scooter images, rental contracts, and report exports stored with SSE-S3 AES-256 encryption.
Email / SMS	AWS SES + AWS SNS	SES for booking confirmation emails; SNS for SMS OTPs and speed violation alerts.
Load Balancer	AWS Application Load Balancer	Distributes traffic across EC2 instances; terminates TLS; routes /api/* to backend, /* to frontend.
Security	AWS WAF	Filters common web attack patterns (XSS, SQL injection, CSRF) before traffic reaches the ALB.
Monitoring	AWS CloudWatch	CPU, memory, and error-rate alarms; log groups for application and access logs; audit trail.
Container Registry	AWS ECR	Docker images stored and versioned; images pulled by EC2 instances during deployment.
Secrets	AWS Secrets Manager + KMS	JWT RSA keys, database credentials, and thirdparty API keys managed securely.

Deployed Application: A working instance of the Scooter Rent Kandy system has been deployed and is accessible at the following link. The table above describes the planned AWS hosting design; the deployed instance may use a simplified hosting setup.

Live System: http://65.0.5.214/

3.7.2 Environment Specification

Attribute	Development	Staging	Production
Infrastructure	Local machine / Docker Compose	AWS EC2 t3.small (single instance)	AWS EC2 t3.medium (Auto Scaling 1–4)
Database	Local PostgreSQL 16	RDS PostgreSQL 16 (single-AZ)	RDS PostgreSQL 16 (Multi-AZ)
Branch	feature/* branches	develop branch	main branch
Logging	Full stack traces enabled	INFO-level logging	ERROR level only
HTTPS	Self-signed certificate	ACM certificate (staging domain)	ACM certificate (production domain)
External APIs	Mock / test mode	Stripe test keys; Twilio test SID	Live production API keys

3.7.3 Network Diagram

The network architecture places all backend resources within an Amazon VPC. EC2 instances and the RDS cluster reside in private subnets with no direct internet access. The Application Load Balancer is deployed in public subnets and is the only entry point for external traffic. NAT Gateways in each Availability Zone allow backend instances to make outbound API calls without being publicly reachable.

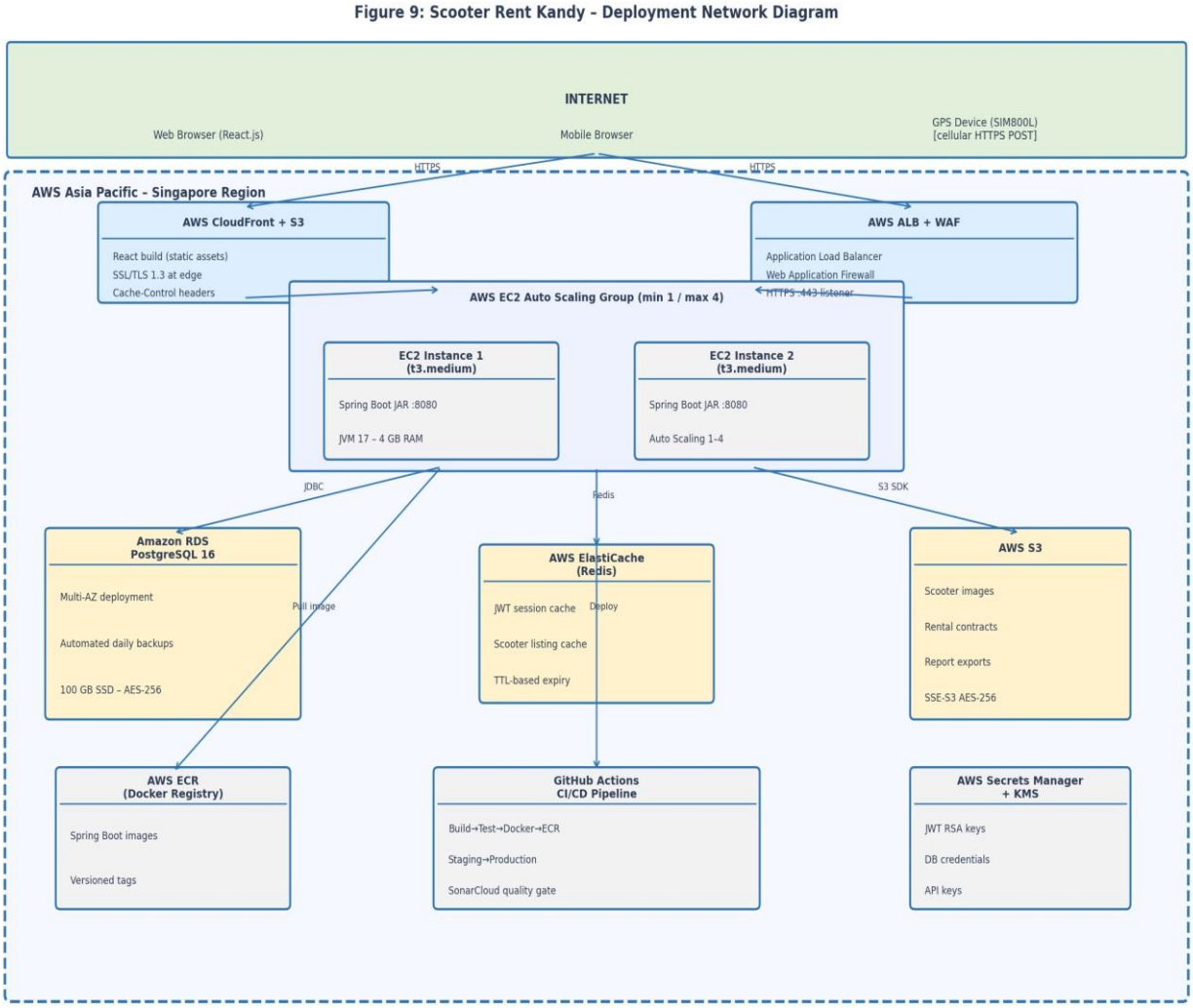


Figure 9: Scooter Rent Kandy – Deployment Network Diagram

3.7.4 CI/CD Pipeline Overview

A GitHub Actions pipeline is employed for continuous integration and delivery, triggered on every push to the develop and main branches.

- Stage 1 – Build & Test: Maven builds the Spring Boot JAR; JUnit 5 unit and integration tests execute; React components tested with Jest. Minimum 80% code coverage enforced.
- Stage 2 – Code Quality: SonarCloud static analysis checks for code smells, duplication, and security vulnerabilities. The build fails if critical issues are detected.
- Stage 3 – Docker Build: A multi-stage Docker image is built from the Spring Boot JAR and pushed to Amazon ECR with a version tag.
- Stage 4 – Deploy to Staging: The new image is deployed to the staging EC2 instance; automated smoke tests execute against the staging URL.
- Stage 5 – Deploy to Production: On merge to main, a manual approval gate is required before deployment. A rolling update strategy ensures zero downtime.

3.8: Data Requirements

3.8.1 Overview

This chapter specifies the data requirements of the Scooter Rent Kandy system. Each data entity, its attributes, data types, constraints, example values, and data formats are described. These requirements are derived directly from the functional requirements and database design presented in Chapter 2.

3.8.2 Data Entities and Attributes

3.8.2.1 User Data

Attribute	Data Type	Constraint	Format / Example	Notes
userId	UUID	PK, NOT NULL	550e8400-e29b-...	Systemgenerated key
firstName	VARCHAR(100)	NOT NULL	Jojeevan	User given name
lastName	VARCHAR(100)	NOT NULL	Arumugam	User family name
email	VARCHAR(255)	NOT NULL, UNIQUE	user@example.com	Login credential
passwordHash	VARCHAR(255)	NOT NULL	$2a$12$…	BCrypt hash; never plain text
phone	VARCHAR(20)	NOT NULL	+94771234567	E.164 international format
nicNumber	VARCHAR(20)	NOT NULL, UNIQUE	200012345678V	AES-256 encrypted at rest
role	ENUM	NOT NULL	USER | ADMIN	RBAC access level
isActive	BOOLEAN	DEFAULT TRUE	true / false	FALSE = account blocked

createdAt	TIMESTAMP	NOT NULL	2026-04-01T10:30:00Z	ISO 8601 UTC

3.8.2.2 Scooter Data

Attribute	Data Type	Constraint	Format / Example	Notes
scooterId	UUID	PK, NOT NULL	uuid-string	Systemgenerated
model	VARCHAR(150)	NOT NULL	Honda Dio	Scooter model name
registrationNo	VARCHAR(30)	NOT NULL, UNIQUE	WP CAA 1234	Sri Lanka vehicle plate
status	ENUM	NOT NULL	AVAILABLE	Real-time fleet status
hourlyRate	DECIMAL(10,2)	NOT NULL, > 0	250.00	LKR per hour
perKmRate	DECIMAL(10,2)	NOT NULL, >= 0	15.00	LKR per km
imageUrl	VARCHAR(500)	NULLABLE	https://s3…./img.png	AWS S3 public URL
totalMileage	DECIMAL(10,2)	DEFAULT 0	12500.50	Cumulative km
lastServiceDate	DATE	NULLABLE	2026-03-15	ISO 8601 YYYYMM-DD

3.8.2.3 Booking Data

Attribute	Data Type	Constraint	Format / Example	Notes
bookingId	UUID	PK, NOT NULL	uuid-string	Systemgenerated

userId	UUID	FK → User, NOT NULL	uuid-string	References User entity
scooterId	UUID	FK → Scooter, NOT NULL	uuid-string	References Scooter entity
startTime	TIMESTAMP	NOT NULL	2026-04-10T09:00:00Z	ISO 8601 UTC
endTime	TIMESTAMP	NULLABLE	2026-04-10T12:00:00Z	NULL if rental is ongoing
pickupLocation	VARCHAR(300)	NOT NULL	Kandy Lake, Kandy, Sri Lanka	GPS address string
dropLocation	VARCHAR(300)	NULLABLE	Peradeniya Road, Kandy	NULL until rental ends
status	ENUM	NOT NULL	ACTIVE	Booking lifecycle state
totalCost	DECIMAL(10,2)	NULLABLE	875.00	Calculated on completion in LKR
contractSigned	BOOLEAN	DEFAULT FALSE	true / false	Must be TRUE to activate

3.8.2.4 Payment Data

Attribute	Data Type	Constraint	Format / Example	Notes
paymentId	UUID	PK, NOT NULL	uuid-string	Systemgenerated
bookingId	UUID	FK → Booking, UNIQUE	uuid-string	One payment per booking
stripePaymentId	VARCHAR(255)	NULLABLE	pi_3NxYZ…	Stripe transaction reference

amount	DECIMAL(10,2)	NOT NULL, > 0	875.00	Total amount in LKR
method	ENUM	NOT NULL	CARD	Payment method used
status	ENUM	NOT NULL	SUCCESS	Payment lifecycle state
paidAt	TIMESTAMP	NULLABLE	2026-04-10T09:05:00Z	NULL until payment succeeds

8.2.5 GPS Log Data

Attribute	Data Type	Constraint	Format / Example	Notes
logId	UUID	PK, NOT NULL	uuid-string	System-generated
bookingId	UUID	FK → Booking, NOT NULL	uuid-string	Links log to active booking
latitude	DECIMAL(9,6)	NOT NULL	7.290572	WGS84 decimal degrees
longitude	DECIMAL(9,6)	NOT NULL	80.633728	WGS84 decimal degrees
speed	DECIMAL(5,2)	NOT NULL, >= 0	45.20	Speed in km/h at log time
recordedAt	TIMESTAMP	NOT NULL	2026-04-10T09:15:00Z	ISO 8601 UTC; every 30 s

3.8.2.6 Camping Gear Data

Attribute	Data Type	Constraint	Format / Example	Notes
gearId	UUID	PK, NOT NULL	uuid-string	System-generated

name	VARCHAR(200)	NOT NULL	Camping Tent	Gear item name
dailyRate	DECIMAL(10,2)	NOT NULL, > 0	500.00	LKR per day
stock	INTEGER	NOT NULL, >= 0	10	Available quantity

3.8.3 Data Volume and Retention

Entity	Est. Records (Year 1)	Growth Rate	Retention Policy
User	~500 registered users	~200 per year	Retained indefinitely; deactivated accounts archived after 2 years
Scooter	~20–30 fleet units	~5 per year	Retained permanently; soft-deleted on disposal
Booking	~5,000 bookings	~4,000 per year	Active records kept 1 year; older records archived automatically
Payment	~5,000 payments	~4,000 per year	Retained for 7 years for financial audit compliance
GPS Log	~1.5 million entries	~1.2 M per year	Retained for 6 months; older logs purged by scheduled job
Camping Gear	~10–20 items	Minimal	Retained indefinitely

3.8.4 Data Formats and Standards

- All timestamps are stored and transmitted in ISO 8601 UTC format (e.g., 2026-0410T09:00:00Z).
- All monetary values are stored as DECIMAL(10,2) representing Sri Lankan Rupees (LKR) to two decimal places.
- GPS coordinates are stored as DECIMAL(9,6) following the WGS84 standard (World Geodetic System 1984).
- All API request and response payloads use the JSON data format with camelCase field naming conventions.

- All date-only fields (e.g., lastServiceDate) use the ISO 8601 date format: YYYY-
    - MM-DD.
- Image files stored in AWS S3 shall be in JPEG or PNG format with a maximum file size of 5 MB per image.
- Exported reports are available in PDF and CSV formats.

3.8.5 Data Privacy and Compliance

- Personal data (name, email, phone, NIC number) is collected only with explicit user consent at the point of registration.
- NIC numbers and payment tokens are encrypted at the application layer using AES256 before database storage.
- User passwords are never stored in plain text; BCrypt hashing with a cost factor of 12 is applied at all times.
- Users may request deletion or export of their personal data in accordance with applicable data protection regulations.
- Payment card data is never stored on the system; Stripe tokenization is used to ensure PCI DSS compliance.
- Audit log entries (user actions, admin activities, login events) are retained for a minimum of one year.

References

[#]	Reference
[1]	IEEE, "IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions," IEEE, 2009.
[2]	IEEE, "IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications," IEEE, 1998.
[3]	ISO/IEC, "ISO/IEC 25010: System and Software Quality Models," ISO/IEC, 2011.
[4]	R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed. New York: McGraw-Hill, 2014.
[5]	I. Sommerville, Software Engineering, 10th ed. Harlow: Pearson, 2016.
[6]	Spring Boot Documentation, "Spring Boot Reference Documentation," [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/. [Accessed: Apr. 2026].
[7]	React Documentation, "React – A JavaScript library for building user interfaces," [Online]. Available: https://react.dev/. [Accessed: Apr. 2026].
[8]	PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," [Online]. Available: https://www.postgresql.org/docs/16/. [Accessed: Apr. 2026].
[9]	Google, "Google Maps Platform – Distance Matrix API," [Online]. Available: https://developers.google.com/maps/documentation/distance-matrix. [Accessed: Apr. 2026].
[10]	Stripe, "Stripe API Reference," [Online]. Available: https://stripe.com/docs/api. [Accessed: Apr. 2026].
[11]	Amazon Web Services, "AWS Well-Architected Framework," [Online]. Available: https://aws.amazon.com/architecture/well-architected/. [Accessed: Apr. 2026].
[12]	W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: Apr. 2026].
[13]	OWASP, "OWASP Top Ten," [Online]. Available: https://owasp.org/wwwproject-top-ten/. [Accessed: Apr. 2026].

Chapter 4

Prototype Development & Client Validation

4.1 UI/UX Design

This section presents the UI/UX design of the Scooter Rent Kandy system, including wireframe descriptions for key user interfaces and the navigation flow prototype. The design was developed in Figma and validated with the client during the prototype demonstration session on 15 April 2026.

4.1.1 Design Principles

The Scooter Rent Kandy interface was designed following five core principles: (1) Consistency – common components reused across all screens with identical styling; (2) Feedback – every action receives immediate visual feedback via loading spinners, success toasts, and error banners; (3) Simplicity – each screen focuses on a single primary task; (4) Accessibility – WCAG 2.1 Level AA compliance throughout; (5) Responsiveness – adapts to desktop (1280px+), tablet (768px), and mobile (375px) viewports.

4.1.2 Key Screen Wireframes

Table 27 summarises the key screens designed for the Scooter Rent Kandy system, their target user role, and the primary UI elements on each screen.

5.2 Screen Descriptions and Accessibility Considerations

Table 17: Screen Descriptions and Accessibility

Screen / Module	Key UI Elements	Accessibility Considerations
Registration / Login	Email input, password field, OTP field, forgot-password link	ARIA labels on all inputs; error messages via aria-live; color contrast ratio ≥ 4.5:1 (WCAG 2.1 AA)

Screen / Module	Key UI Elements	Accessibility Considerations
Scooter Browse & Search	Grid of scooter cards with image, model, price/hr, availability badge; filter sidebar	Keyboard-navigable filter panel; descriptive alt text on scooter images; visible focus indicators
Booking Wizard	Date-time picker, camping gear checklist, Google Maps pickup selector, booking summary	Step progress bar with ARIA progress role; form fields grouped with <fieldset> and <legend>

Screen / Module	Key UI Elements	Accessibility Considerations
Digital Contract	Inline PDF viewer, signature canvas, accept/decline buttons	PDF viewer with zoom controls; text-based fallback for motor-impaired users
Payment Page	Stripe embedded card element, order summary, secure payment badge	Stripe Elements are fully accessible; inline error messages below each field

Screen / Module	Key UI Elements	Accessibility Considerations
User Dashboard	Active bookings list, booking history table, notification bell, profile settings	Tables with proper <th> scope; notification count exposed via aria-label
GPS Tracking View	Google Maps with real-time scooter marker, distance counter, speed indicator	Text-based status panel for non-visual users; speed indicator uses color + numeric value

Screen / Module	Key UI Elements	Accessibility Considerations
Admin Dashboard	Revenue chart, fleet status grid, booking queue, speed violation alerts	Charts with data table alternative; alerts use aria-live=polite announcements
Admin Reports	Date range filter, export buttons, sortable data tables	All export buttons keyboard-accessible; tables sortable with ARIA sort attributes

Screen / Module	Key UI Elements	Accessibility Considerations

Table 27: Key Screen Wireframes and UI Elements

Screen	Role	Key UI Elements
Landing / Home Page	Public	Hero banner, "Book a Scooter" CTA, scooter listing cards, search bar
Registration Page	Public	Name, email, phone, NIC, password form with strength indicator
Login Page	User / Admin	Email and password form; "Forgot Password" link
Scooter Browse Page	User	Grid of scooter cards with image, model, price/hr, availability badge
Booking Wizard	User	Date-time picker, Google Maps pickup selector, camping gear add-ons, digital contract
Payment Page	User	Stripe embedded card element, order summary sidebar with cost breakdown
Booking History	User	Timeline with status badges; Cancel and View Receipt actions
GPS Tracking View	User	Google Maps real-time scooter marker, distance counter, speed indicator
Admin Dashboard	Admin	KPI cards: active rentals, daily revenue, fleet status; GPS map view
Fleet Management	Admin	Scooter data table with inline edit; Add Scooter modal with image upload
Reports Page	Admin	Date-range filter; export PDF/CSV; revenue and mileage charts

4.1.3 Prototype Navigation Flow

The navigation flow prototype was developed in Figma and demonstrates the complete interaction paths for both User and Admin roles. The prototype covers the following key flows: (1) user registration and login; (2) scooter browsing and booking with digital contract acceptance; (3) GPS-tracked active rental and automated billing; (4) admin fleet management and report generation. Role-based routing is enforced throughout: unauthenticated users are redirected to /login; unauthorized access returns HTTP 403 Forbidden.

The prototype was shared with the client during the demonstration session on 15 April 2026. The client confirmed that the interface layout is intuitive and meets the business requirements specified in the approved SRS.

4.2 Client Involvement Section

This section documents the client involvement activities conducted during the design and prototype phases of the project. Two formal engagement sessions were held with the client representative, Mr. Tharindu Rajapakshe, of Scooter Rent Kandy.

4.2.1 Client Demonstration Session

A formal prototype demonstration session was conducted with the client on 15 April 2026. During this session, the development team demonstrated the Figma prototype covering all major user flows including scooter browsing, the booking wizard, the GPS tracking view, the payment page, and the admin dashboard. All four team members and the client representative attended. Meeting minutes were recorded and signed by the client, confirming the demonstration.

4.2.2 Client Feedback Report

Following the demonstration session, the client provided the following written feedback:

- The overall layout and navigation flow are intuitive and straightforward for rental customers.
- The GPS-based billing display on the active rental screen clearly communicates the current distance and estimated cost.
- The admin dashboard KPI cards provide sufficient visibility into daily operations.
- The client requested that scooter images on the Browse page be displayed with larger thumbnails to improve visual appeal. This change was actioned in the updated prototype.
- The client confirmed that the digital contract acceptance step adequately protects the business from liability disputes.

All client feedback was reviewed by the development team. The image thumbnail size was updated in the prototype accordingly. A formal sign-off confirming client approval is provided in Section 4.5.

4.2.3 Official Client Feedback Proof

APPENDIX D: CLIENT FEEDBACK AND APPROVAL

Date: April 30, 2026

From:
Tharindu Rajapakshe
Owner/Founder, Scooter Rent Kandy
Kandy, Sri Lanka

To:
The Project Development Team
Software Engineering Teaching Unit
Faculty of Science, University of Kelaniya

Subject: Feedback and Approval of SDS and SRS Documentation – Scooter Rent Kandy

Dear Project Team,

I am writing to provide my formal feedback regarding the Software Requirement Specification (SRS) and System Design Specification (SDS) documents you submitted for the 'Scooter Rent Kandy' management system.

First, I would like to commend the team on the quality of the work. It is evident that a significant amount of effort has gone into the technical architecture and the planning of the mobile tracking features. The work is good, and I am satisfied with the progress made thus far.

However, I must note that as a business owner, I found the documents somewhat difficult to fully grasp due to their highly technical nature. While the system's logic is solid, it was a bit of a challenge for me to get a clear and simple overview of the entire process from a non-technical perspective.

- Moving forward, I suggest the following:
- Please continue with the development as planned, as I am happy with the project's direction.
- If possible, include some simplified summaries or visual guides that help me visualize the user journey more easily.
- Please let me know if you require any further information, resources, or specific business data from my end to assist with the next phase.

I look forward to seeing the next iteration of the system.

Sincerely,





Tharindu Rajapakshe
Owner, Scooter Rent Kandy

4.2.3 Official Client Feedback – SRS and SDS Review

The following records the official feedback from Mr. Tharindu Rajapakshe upon his review of Version 1.0 of the SRS and SDS, received on 30 April 2026.

Field	Details
Date of Feedback	April 30, 2026
Stakeholder	Tharindu Rajapakshe – Client, Scooter Rent Kandy
Documents Reviewed	SRS Version 1.0 and SDS Version 1.0
Communication Channel	In-person meeting with verbal and written confirmation
Recorded By	Development Team (SE/2021/013, 032, 045, 048)

Table 28: Official Client Feedback – Metadata

Summary: The client expressed overall satisfaction with the quality and direction of the work, providing a green light to proceed. His key observations were: (1) Approval – "The work is good. Let's continue it." (formal verbal endorsement); (2) Technical complexity – the depth of the SRS/SDS makes conceptual understanding difficult for a non-technical reviewer (constructive, not a rejection); (3) Open collaboration – the client offered to provide any further resources needed.

Action Plan in response to feedback:

#	Action	Description	Target
1	Executive Summaries	Add a Plain English summary at the start of Chapters 2 and 3 explaining the system without technical terminology.	Ch 2 & 3
2	Enhanced Visualisation	Add high-level user journey flowcharts illustrating the path from browsing through booking, payment, and GPS rental.	Ch 2 & 3
3	Glossary of Terms	Include a glossary in the Appendix defining key terms: NFR, API, GPS, JWT, RBAC, BCNF, ORM.	Appendix

Table 29: Client Feedback Action Plan

4.3 Ethical Considerations

The Scooter Rent Kandy project was developed with careful consideration of the ethical responsibilities associated with software systems that handle personal data, financial transactions, and physical safety.

4.3.1 Data Privacy and Consent

- Personal data (name, email, phone, NIC) is collected only with explicit user consent at registration.
- NIC numbers are encrypted at the application layer using AES-256 before storage. User passwords are BCrypt-hashed (cost factor 12); plain text is never stored.
- Payment card data is never stored on the system. Stripe tokenisation is used for all payment processing to ensure PCI DSS compliance.
- Users retain the right to request deletion or export of their personal data in accordance with applicable data protection regulations.

4.3.2 Safety and Accountability

- The GPS speed monitoring feature alerts the system when a scooter exceeds 60 km/h. Speed violation events are logged with timestamps, GPS coordinates, and user identifiers.
- The digital contract acceptance step ensures that users formally acknowledge the terms and conditions of the rental, including responsibilities for safe operation.
- Audit log entries are retained for a minimum of one year to support post-incident investigations.

4.3.3 Accessibility and Inclusivity

- The system interface meets WCAG 2.1 Level AA accessibility guidelines, ensuring usability for individuals with visual, motor, or cognitive impairments.
- The system supports three languages (English, Sinhala, and Tamil) to ensure inclusivity for the diverse user population in the Kandy region.
- All form inputs include descriptive labels and accessible error messages via ARIA live regions.

4.3.4 Academic Integrity

This project was developed solely for academic purposes. All code, documentation, and design artefacts are original work by the development team. All third-party libraries, frameworks, and APIs used have been properly acknowledged. The client agreement explicitly states that no commercial exploitation will occur without written consent of the University of Kelaniya.

4.4 Reflection on Stakeholder Communication

Effective stakeholder communication was central to the success of the Scooter Rent Kandy project. Three formal channels were used: in-person meetings for requirement elicitation and prototype demonstrations, written correspondence (email) for formal approvals, and signed meeting minutes for all formal sessions.

4.4.1 Communication Approach

The initial requirement elicitation meeting was conducted in February 2026. A second meeting in March 2026 validated the SRS and obtained formal sign-off. The prototype demonstration session in April 2026 completed the primary client engagement cycle. On 30 April 2026, the client reviewed and approved the SRS and SDS documents, confirming no mandatory revisions were required.

4.4.2 Challenges Encountered

- Scheduling formal meetings with the client was occasionally challenging due to business operating hours. This was resolved by offering flexible meeting times and supplementing in-person sessions with email correspondence.
- Some initial requirements were ambiguous, particularly the GPS billing calculation logic. These were resolved through targeted follow-up questions and explicit acceptance criteria in the SRS.
- The client's unfamiliarity with technical terminology required translating design concepts into business language. Visual mockups and the live Figma prototype significantly improved comprehension.

4.4.3 Lessons Learnt

- Establishing a formal communication protocol (meeting minutes, email confirmations, signed sign-offs) from the outset prevented misunderstandings and provided a clear audit trail.
- Iterative client feedback on design artefacts proved more effective than document-based review alone, as the client could interact directly with the proposed interface.
- Maintaining transparent communication about project constraints and timeline limitations built trust and ensured realistic client expectations.

4.5 Formal Sign-off from the Client for the Development

The following sign-off confirms that the client representative has reviewed and approved the prototype and design artefacts, authorising the development team to proceed with the implementation phase.

Field	Details
Project Title	Scooter Rental Management System – Scooter Rent Kandy
Client Organisation	Scooter Rent Kandy, Kandy, Sri Lanka
Client Representative	Mr. Tharindu Rajapakshe
Approved Deliverables	SRS, SDS, Figma Prototype and Navigation Flow
Date of Approval	15 April 2026
Approval Method	Written sign-off following prototype demonstration session

Table 30: Client Sign-off Details

Client Representative Sign-off

Name: Tharindu Rajapakshe

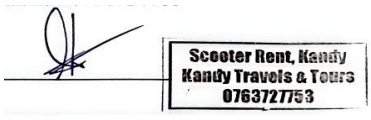
Title: Business Owner

Organization: Scooter Rent Kandy

Date: 15 April 2026

Signature:

Official Seal:



Project Supervisor Sign-off

Name: Mrs. Nimesha Arambepola

Date: 25/10/2026

Signature: _____

Note: Any changes to the approved design after this sign-off must be submitted as a formal Change Request and re-approved by both parties before implementation commences.

Chapter 5

Conclusion

The final chapter summarizes the outcomes of the **Scooter Rent Kandy** project and evaluates how successfully the proposed objectives were achieved. This chapter focuses on the overall performance of the system in terms of usability, accessibility, reliability, and user friendliness. It also discusses the limitations and drawbacks identified during development and testing. In addition, future modifications, improvements, and possible extensions are presented to support the continued growth of the system. Finally, this chapter concludes the development of the Scooter Rent Kandy system as an efficient digital solution for scooter rental services in Kandy.

5.1 Degree of Objectives Met

Overall, the objectives identified at the beginning of the project have been successfully achieved. The system allows customers to browse available scooters, make reservations, and manage bookings through an easy-to-use online platform. Customers can select rental dates, view scooter details, and complete bookings efficiently.
The admin can manage scooter availability, rental pricing, customer records, and booking confirmations through the administrative dashboard. Secure login features ensure that only authorized users can access sensitive functions.
The project has successfully met its goal of modernizing traditional scooter rental operations by replacing manual processes with a reliable web-based management system.

5.2 Usability, Accessibility, Reliability and Friendliness

Usability
Usability was a major priority in the development of the Scooter Rent Kandy system. The interface is designed in a clear and organized manner, allowing users to navigate easily through scooter listings, booking forms, and account features. Important actions such as booking, registration, and viewing rental history can be completed with minimal effort.
Accessibility
The system is designed to be accessible through modern web browsers on desktops, laptops, tablets, and mobile devices. Responsive layouts help users access the platform from different screen sizes. Clear fonts, readable colors, and structured page layouts improve accessibility for a wide range of users.
Reliability
Reliability is essential for a rental management system. The Scooter Rent Kandy platform stores booking records securely and reduces the possibility of double bookings through proper availability management. User authentication, database consistency, and tested system modules help ensure dependable performance.
Friendliness

The user-friendly nature of the system is reflected in its simple interface, which empowers non-technical fleet managers to oversee complex operations with minimal training. Customers can quickly search scooters, reserve vehicles, and receive booking confirmations with ease. The admin panel provides straightforward management tools, making day-to-day rental operations more efficient and reliable.

5.3 Limitations and Drawbacks

Although the project was successfully completed, some limitations remain.

- Online payments may depend on third-party payment gateway integration.
- Internet access is required to use the system effectively.
- The system currently focuses mainly on scooter rentals within the Kandy area.
- Real-time GPS tracking of scooters is not yet implemented.
- Advanced analytics and automated reporting features are limited.
- Multi-language support is currently minimal.

These limitations provide opportunities for future development.

5.4 Timeline

The project was completed according to the planned development schedule. Major phases included requirement gathering, system analysis, design, implementation, testing, documentation, and final deployment preparation. Each phase was completed within the allocated timeframe, allowing the project objectives to be achieved successfully.

5.5 Future Modifications, Improvements and Extensions Possible

Several future enhancements can further improve the Scooter Rent Kandy system:

- **Online Payment Integration** – Add secure card payments and digital wallet options.
- **GPS Tracking** – Enable live scooter tracking and route monitoring.
- **Mobile Application** – Develop Android and iOS apps for easier booking access.
- **Multi-Language Support** – Add Sinhala, Tamil, and English language options.
- **Customer Reviews and Ratings** – Allow users to rate scooters and services.
- **AI Recommendations** – Suggest scooters based on customer preferences and booking history.

Native Mobile Application – Develop dedicated Android and iOS native apps using React Native, providing offline booking capabilities, push notifications, and an optimised mobile-first experience beyond the current responsive web interface.

AI-Driven Price Optimisation – Implement machine learning algorithms to dynamically adjust rental pricing based on demand patterns, fleet utilisation rates, seasonal trends, and competitor pricing, maximising revenue while maintaining competitive rates for customers.

- **Advanced Reports** – Provide detailed rental, income, and customer trend reports.
- **Expansion to Other Cities** – Extend services beyond Kandy to other tourist areas in Sri Lanka.

By implementing these future improvements, the Scooter Rent Kandy system can become a more advanced, competitive, and customer-friendly rental platform.

5.6 Deployed System

The Scooter Rent Kandy system has been deployed and is publicly accessible at the link below.

http://65.0.5.214/

Appendix B: Project Poster

The following full-page poster was prepared by the development team for presentation at the Software Engineering Teaching Unit, Faculty of Science, University of Kelaniya. The poster summarises the Scooter Rent Kandy system objectives, technology stack, business model, and team composition.

Team Members:

- Jojeevan – SE/2021/013
- Kajanan – SE/2021/032
- Gobinath – SE/2021/045
- Mohamad – SE/2021/048



Figure B.1: Scooter Rent Kandy – Project Poster (Faculty of Science, University of Kelaniya)

Appendix C: References (IEEE Format)

All references cited in this report are formatted according to the IEEE citation standard. Sources are numbered sequentially in the order they are cited in the text.

[#]	Reference (IEEE Format)
[1]	IEEE, "IEEE Std 1016-2009: IEEE Standard for Information Technology – Systems Design – Software Design Descriptions," IEEE, 2009.
[2]	IEEE, "IEEE Std 830-1998: Recommended Practice for Software Requirements Specifications," IEEE, 1998.
[3]	ISO/IEC, "ISO/IEC 25010:2011 – System and Software Quality Models," ISO/IEC, 2011.
[4]	R. S. Pressman, Software Engineering: A Practitioner's Approach, 8th ed. New York, NY, USA: McGraw-Hill, 2014.
[5]	I. Sommerville, Software Engineering, 10th ed. Harlow, UK: Pearson Education, 2016.
[6]	Spring Boot, "Spring Boot Reference Documentation," [Online]. Available: https://docs.spring.io/spring-boot/docs/current/reference/html/. [Accessed: Apr. 2026].
[7]	React, "React – A JavaScript Library for Building User Interfaces," [Online]. Available: https://react.dev/. [Accessed: Apr. 2026].
[8]	PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," [Online]. Available: https://www.postgresql.org/docs/16/. [Accessed: Apr. 2026].
[9]	Google, "Google Maps Platform – Distance Matrix API," [Online]. Available: https://developers.google.com/maps/documentation/distance-matrix. [Accessed: Apr. 2026].
[10]	Stripe, "Stripe Payments API Reference," [Online]. Available: https://stripe.com/docs/api. [Accessed: Apr. 2026].
[11]	Amazon Web Services, "AWS Well-Architected Framework," [Online]. Available: https://aws.amazon.com/architecture/well-architected/. [Accessed: Apr. 2026].
[12]	W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," [Online]. Available: https://www.w3.org/TR/WCAG21/. [Accessed: Apr. 2026].
[13]	OWASP, "OWASP Top Ten Web Application Security Risks," [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Apr. 2026].

Table 34: References (IEEE Format)

Proofreading Certification

I hereby certify that I have carefully proofread the final draft of this report titled “Final Design Report” and have checked for grammatical, spelling, and language accuracy to the best of my knowledge.

To the best of my ability, I confirm that the document is free from major grammatical and spelling errors. However, any remaining errors are the sole responsibility of the student.

Name of Certifier: Thavalingam Kajaluxshan

Designation: Associate Software Engineer,iVedha .

Signature: 

Date: 25/10/2026